prompt1: ROLE

Senior platform/security engineer.

GOAL

Add production-grade HSTS on frontend + staged rollout to preload-ready.

AUDIT

2026-03-03-full-system-audit: missing HSTS on frontend.

RULES

- Do NOT set includeSubDomains/preload until ALL subdomains are HTTPS-only and listed.
- Single source of truth for STS headers (Pages OR Worker). No duplicates.
- Rollout must be safe: short max-age first.

TASKS

1) DISCOVER (no guessing)

- List all active hostnames in zone (DNS + Pages/Workers routes).
- Mark which are user-facing.
- For each hostname:
  - HTTPS works (valid cert)
  - HTTP redirects 301 → HTTPS
  - Record results in a small table.

2) IMPLEMENT STAGE A (primary frontend only)

Set STS on primary frontend hostname(s) ONLY:

- Start: max-age=300
- Then: max-age=86400 after verification
- Then: max-age=31536000 after 48h stable

3) STAGE B (only if proven safe)

If every subdomain is HTTPS-only + 301 redirect:

- add includeSubDomains

4) STAGE C (only if proven safe)

If B is true:

- add preload
- final header must be exactly:

  Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
- Provide preload submission steps (no links).

5) APPLY VIA CLOUDFLARE (choose correctly)

- If Cloudflare Pages: implement via `_headers` for `/*`
- If Cloudflare Worker: inject header in fetch handler for ALL responses:
  - success + errors + redirects
  - avoid duplicates (if already set, keep strongest value)
- If both exist: pick ONE and remove/disable the other's STS.

6) VERIFY (copy-paste commands)

Provide curl checks for:

- https://<primary>/

- https://<primary>/<deep-route>

- https://<primary>/<static-asset>

- http://<primary>/ (must 301 to https and still include STS on final https response)

Checks:

- STS header present

- no duplicated STS

- max-age matches current stage

7) MONITOR

Provide 1 synthetic check idea to alert if STS disappears.

8) ROLLBACK

- Steps to remove STS quickly (Cloudflare config revert).

- Note browser caching impact (can’t instantly undo for clients with high max-age).

- Keep ramp strategy (300 → 86400 → 31536000).

OUTPUT ONLY (no explanations)

A) SUMMARY (≤5 bullets)

B) CHANGES (file paths + code blocks)

C) CURL_TESTS (commands + expected key headers)

D) GATES (yes/no criteria for includeSubDomains + preload)

E) ROLLBACK_STEPS

prompt2: ROLE

Senior platform engineer.

GOAL

Fix API CORS in production and add Cloudflare edge protection.

AUDIT

2026-03-03-full-system-audit:

- localhost CORS leak

- preflight mismatch

- missing Cloudflare proxy/WAF

- no edge rate limiting

INFRA

Cloudflare → Hetzner origin

API domain: https://api.lap-coach.racing

Frontend domains include:

- https://lap-coach.racing

- https://lap-coach.racing/app

RULES

- Never return ACAO localhost in PROD

- No wildcard origin

- Same logic for OPTIONS + normal requests

- One CORS source only

- Allowed origins defined via env

ENV VAR

CORS_ALLOWED_ORIGINS=comma list

TASKS

1. FIND CORS

Locate where CORS headers are set.

Remove duplicates (proxy/app).

2. IMPLEMENT CORS

Single middleware:

- allow origin only if in allowlist

- set:

  Access-Control-Allow-Origin

  Access-Control-Allow-Credentials

  Access-Control-Allow-Methods

  Access-Control-Allow-Headers

  Access-Control-Max-Age

  Vary: Origin

- support OPTIONS preflight

3. APPLY SAME LOGIC

Apply identical rules to:

OPTIONS

GET/POST/etc

error responses

4. PROD GUARD

Add CI check:

PROD config cannot include localhost.

5. TESTS

Add tests:

allowed origin → ACAO present

blocked origin → no ACAO

preflight headers correct

6. CLOUDFLARE PROXY

Ensure api domain is proxied (orange cloud).

SSL mode: Full (strict).

7. WAF

Enable Cloudflare managed OWASP rules.

8. RATE LIMITS

Create rules:

auth endpoints → strict

compute endpoints → medium

global API → moderate

Return HTTP 429.

9. BOT PROTECTION

Enable Cloudflare bot protection for auth endpoints.

10. EXCEPTIONS

Do not block:

Stripe webhooks

health checks

uptime monitors

11. ORIGIN LOCKDOWN

Configure Hetzner firewall:

allow only Cloudflare IP ranges to API port.

12. VERIFY

Provide curl commands:

Allowed origin

curl -H "Origin: https://lap-coach.racing"

Preflight

curl -X OPTIONS ...

Blocked origin

Origin: http://evil.com

Checks:

ACAO correct

Vary header present

no localhost.

13. OUTPUT FORMAT

Return only:

FILES_CHANGED

CLOUDFLARE_RULES

FIREWALL_RULES

CURL_TESTS

ROLLBACK_STEPS

No explanations.

No commentary.

prompt 3: ROLE Senior platform/security engineer GOAL Harden Lap platform security: enforce modern TLS, remove insecure defaults, add secret management discipline, and implement automated supply-chain security controls. AUDIT 2026-03-03-full-system-audit findings include: TLS not pinned to modern minimum in Caddy, Flask default secret key present ("telemetry-local-dev-key"), no secret rotation process, no CI secret scanning, no automated dependency updates, no automated vulnerability scanning. INFRA Cloudflare edge → Caddy reverse proxy → Flask API backend (Hetzner). RULES Production reliability required. No silent failures. No insecure defaults allowed. All security controls must be automated and verifiable. Minimize config drift. TASKS 1 TLS HARDENING Locate Caddy configuration (Caddyfile or JSON config). Enforce minimum TLS version ≥1.2 and prefer TLS 1.3. Ensure strong cipher defaults. Confirm TLS termination path (Cloudflare edge → origin). Ensure Caddy configuration does not allow TLS 1.0 or 1.1. Apply configuration safely without breaking Cloudflare Full (strict) SSL mode. 2 SECRET KEY SECURITY Locate Flask application configuration. Remove any default fallback secret value including "telemetry-local-dev-key". Application must fail fast at startup if SECRET_KEY is missing in production. Implement environment variable based secret loading. Add runtime validation that aborts startup if SECRET_KEY not set. 3 SECRET MANAGEMENT POLICY Create a secret rotation runbook including: secrets inventory list (API keys, DB credentials, session secrets, Stripe secrets etc.), rotation frequency recommendation (e.g. 90 days), procedure for rotating each secret type, verification steps after rotation. Provide a minimal automated rotation reminder schedule (calendar or CI reminder). 4 CI SECRET SCANNING Add automated secret scanning in CI to prevent commits containing credentials. Configure scanner to block commits containing API keys, tokens, private keys, or secrets. Ensure scanning runs on pull requests and repository history. Fail CI if leaks detected. 5 DEPENDENCY UPDATE AUTOMATION Enable automated dependency update system (Dependabot or Renovate). Configure separate update groups for backend and frontend dependencies. Limit update batch size to avoid massive PRs. Ensure updates run on a regular schedule. 6 AUTOMATED VULNERABILITY SCANNING Add software composition analysis (SCA) scanning in CI for both backend and frontend dependencies. Ensure scans detect known vulnerabilities. CI must fail for critical vulnerabilities. Reports must be visible in pull requests. 7 VERIFICATION Provide commands or CI checks that verify: TLS version enforcement active, application fails to start without SECRET_KEY, CI detects injected test secrets, dependency update automation runs, vulnerability scan runs and produces report. OUTPUT FORMAT Return only: SUMMARY (≤5 bullets) CONFIG_CHANGES (files + code blocks) CI_CHANGES (pipeline config) SECURITY_RUNBOOK (secret rotation steps) VERIFICATION_COMMANDS (commands/tests) ROLLBACK_STEPS No explanations outside these sections.

prompt 4:
ROLE Senior platform/security engineer GOAL Harden Lap platform against client-side injection, supply-chain tampering, webhook replay attacks, admin abuse, data exposure, and information leakage. AUDIT 2026-03-03-full-system-audit findings include: CSP uses unsafe-inline for styles, Chart.js loaded without SRI, Stripe webhooks lack replay protection, admin endpoints lack IP restrictions, Redis exposure risk, downloads missing Content-Disposition attachment, API errors leak DB details. INFRA Cloudflare edge → Caddy →

Flask API → Redis/Postgres. RULES Production-grade security. No insecure defaults. All mitigations must be deterministic, minimal, and verifiable. Avoid breaking production traffic. TASKS 1 CSP HARDENING Locate CSP header definition (Caddy, Worker, or app middleware). Remove style-src 'unsafe-inline'. Replace with nonce or hash approach. Generate nonce per response when inline styles are unavoidable. Prefer eliminating inline styles if possible. Ensure policy still allows required assets and frameworks. 2 CDN INTEGRITY Add Subresource Integrity (SRI) for Chart.js if using CDN. Compute sha384 integrity hash for the exact version used. Add crossorigin attribute. Alternatively self-host Chart.js with pinned version if CDN integrity cannot be maintained. 3 STRIPE WEBHOOK REPLAY PROTECTION Locate Stripe webhook endpoint implementation. Validate Stripe-Signature header using official library. Enforce timestamp tolerance window (default ≤300 seconds). Reject events outside tolerance window. Ensure webhook handler is idempotent (prevent duplicate event processing). 4 ADMIN ACCESS HARDENING Identify admin-only endpoints or admin dashboard routes. Add optional IP allowlist configuration. Allowlist controlled via environment variable ADMIN_ALLOWED_IPS. If enabled, reject requests from non-listed IPs before authentication logic. Ensure Cloudflare real IP headers are respected. 5 REDIS HARDENING Inspect Redis deployment. Ensure Redis is not exposed publicly. Either bind Redis to localhost or requirepass authentication. Confirm Redis port not reachable externally. 6 SAFE FILE DOWNLOADS Locate API endpoints returning files (CSV, exports, telemetry downloads). Add response header Content-Disposition: attachment with safe filename. Prevent browsers from rendering file inline. Ensure proper Content-Type is still sent. 7 ERROR SANITIZATION Review API error handling. Remove database-specific error details (column names, SQL fragments, stack traces). Return generic error messages to clients. Preserve full errors only in server logs. OUTPUT FORMAT Return only: SUMMARY (≤5 bullets) CONFIG_CHANGES (files + code blocks) SECURITY_RULES (CSP, webhook validation, Redis config) VERIFICATION_COMMANDS (curl/tests) ROLLBACK_STEPS No explanations outside these sections.

prompt 5: ROLE Senior application security engineer GOAL Execute a lightweight real penetration test on Lap (external, black-box), produce a prioritized remediation backlog, and set up tracking so fixes are verified and closed. AUDIT 2026-03-03-full-system-audit requires a real pentest + remediation tracking. SCOPE Targets: https://lap-coach.racing (marketing), https://lap-coach.racing/app (webapp), https://api.lap-coach.racing (API). Test from the public internet only (no internal network). Include authenticated testing with 2 roles: standard user and admin (if admin exists). Exclude: DoS beyond safe rate-limits, social engineering, physical attacks. RULES No disruption. Respect rate limits. No data exfiltration beyond proof-of-concept minimal samples. Document everything reproducibly. TASKS 1 PREP Create a test plan with: scope, accounts needed, allowed hours, data handling, contact for emergencies. Define success criteria: OWASP Top 10 coverage + auth/session + access control + upload/export + billing/webhooks + infra edge. 2 TOOLING Run automated scans (safe settings) and manual verification. Use at minimum: Nuclei (templates relevant), OWASP ZAP baseline + authenticated scan, Burp Community workflow (or equivalent), SSL/TLS scan, dependency surface checks from exposed assets. Keep scan settings conservative to avoid overload. 3 TEST AREAS Cover at least: Authentication (bruteforce protections, MFA if any, password reset), Session management (cookies, CSRF, refresh tokens), Access control (IDOR,

horizontal/vertical privilege), CORS/CSP/SRI, XSS (reflected/stored/DOM), SSRF, open redirects, file upload handling (type validation, storage access), download endpoints (content-disposition), API error leakage, rate limiting (login/upload/compute), webhook security (Stripe signature + replay), admin hardening (IP allowlist), Redis exposure, security headers, HTTP methods, cache poisoning risks, cloud edge misconfig (direct origin access), JWT validation pitfalls if used. 4 EVIDENCE For each finding: reproduce steps, request/response snippets, impact, severity, affected endpoints, and a minimal proof (no mass data). Provide remediation guidance and verification steps. 5 REPORT Deliver a pentest report with: executive summary, methodology, findings table (severity, CVSS-like score, OWASP mapping), detailed findings, and retest checklist. 6 TRACKING Create a remediation tracker as: (a) GitHub Issues with labels severity/component/status OR (b) a single markdown file SECURITY_REMEDIATION.md with a table. Each finding must have an owner, due date, fix PR link, and retest status. 7 RETEST After initial fixes, rerun the minimal scan suite and manually retest high/critical findings. Mark findings as fixed only with evidence. OUTPUT FORMAT Return only: TEST_PLAN (bullets) COMMANDS (exact commands with safe flags) MANUAL_CHECKLIST (compact) REPORT_TEMPLATE (headings + tables) TRACKING_SETUP (GitHub labels or file format) RETEST_STEPS No extra commentary.

prompt 6: ROLE Senior billing/entitlements engineer GOAL Fix billing correctness so subscription_status is not always “free”, align plan_tier/subscription_status with Stripe customer+subscription IDs as a single source of truth, and enforce a billing state machine in middleware. AUDIT 2026-03-03-full-system-audit: subscription_status always “free” across tiers. RULES No silent failures. Server-side enforcement only (frontend never trusted). Stripe is source of truth for paid state; local DB may cache but must be consistent + invalidated via webhooks. Changes must be testable with fixtures and reproducible steps. TASKS 1 DISCOVERY Locate current billing fields and logic: plan_tier, subscription_status, stripe_customer_id, stripe_subscription_id, webhook handlers, auth middleware, and any “free” fallback. Identify why status is stuck (missing webhook updates, wrong env keys, wrong lookups, stale cache, bad DB mapping). Produce a minimal diagnosis (1–3 bullets max). 2 DATA MODEL SINGLE SOURCE OF TRUTH Define canonical billing model: - stripe_customer_id (nullable) - stripe_subscription_id (nullable) - stripe_price_id or internal plan_id - billing_state enum: trialing|active|past_due|canceled|unpaid|free - current_period_end, cancel_at_period_end, updated_at Ensure plan_tier is derived from active price/plan mapping, not manually set. Ensure subscription_status is derived from billing_state (or deprecate one). Choose one canonical field (billing_state) and make other fields computed/compat. 3 BILLING STATE MACHINE Implement deterministic mapping from Stripe subscription status + invoice/payment states to billing_state: - no customer/subscription → free - subscription.status=trialing → trialing - active + paid/in_good_standing → active - past_due → past_due - unpaid → unpaid - canceled/incomplete_expired → canceled Define grace policy (if any) but keep states accurate. 4 WEBHOOKS (CONSISTENCY) Ensure webhook processing is idempotent and updates billing_state + plan mapping: - Store webhook event_id unique; ignore duplicates - Handle key events: customer.subscription.created|updated|deleted, invoice.payment_succeeded|failed, checkout.session.completed (if used) - Update stripe_customer_id/subscription_id if missing - Recompute billing_state and plan_tier on each

relevant event - If webhook missing, provide a fallback “sync endpoint” or scheduled reconciliation job (minimal). 5 MIDDLEWARE ENFORCEMENT Add server middleware that enforces entitlements based on billing_state and plan_tier for protected endpoints. Must: - Read billing_state from DB/cache - If stale or missing and stripe_subscription_id exists, optionally perform a lightweight Stripe fetch with strict timeout OR rely on last_synced + background reconcile; choose one and justify in 1 bullet max - Return consistent error envelope with reason codes: PLAN_REQUIRED, PAST_DUE, UNPAID, CANCELED - Never allow paid features if billing_state not active/trialing per policy. 6 TESTS Add tests covering: - free user blocked from paid endpoints - active user allowed - trialing allowed - past_due/unpaid blocked (or limited) per policy - webhook updates transition states correctly - plan_tier derived correctly from price_id mapping - regression test: subscription_status not always “free”. 7 MIGRATION + BACKFILL Provide DB migration if needed + a one-time backfill script: - For users with stripe_customer_id/subscription_id, fetch Stripe and set billing_state/plan_tier - Log summary counts; safe retries. 8 VERIFICATION Provide manual test steps using Stripe test mode: - create checkout session for each tier - complete payment - verify DB fields + API access - simulate payment failure → past_due/unpaid - cancel subscription → canceled - ensure app reflects correctly. OUTPUT FORMAT Return only: SUMMARY (≤5 bullets) CODE_CHANGES (file paths + code blocks) DB_MIGRATION (if any) WEBHOOK_MATRIX (event → updates) STATE_MACHINE (mapping table) MIDDLEWARE_POLICY (allowed/blocked) TESTS (what added) MANUAL_VERIFY_STEPS ROLLBACK_STEPS No extra commentary.

prompt 7:
ROLE Senior billing engineer GOAL Complete Stripe LIVE mode setup and validate an end-to-end upgrade flow (free → paid) in production-grade manner. AUDIT 2026-03-03-full-system-audit: Stripe live mode incomplete / upgrade flow not validated. RULES No silent failures. Stripe is source of truth. Webhooks must be reliable + idempotent. All steps must be verifiable in LIVE (with safe low-cost price) and mirrored in TEST. Never expose secret keys client-side. TASKS 1 INVENTORY + GAP CHECK Locate Stripe integration code/config (frontend + backend). Identify current usage of TEST keys vs LIVE keys, webhook secrets, price IDs, success/cancel URLs, customer portal config, and environment flags. Output a minimal gap list (max 5 bullets). 2 LIVE MODE SETUP Ensure LIVE configuration exists and is wired correctly: - STRIPE_SECRET_KEY (LIVE) - STRIPE_WEBHOOK_SECRET (LIVE endpoint) - STRIPE_PRICE_IDS per plan (LIVE) - STRIPE_PUBLIC_KEY (LIVE) - APP_URL / success_url / cancel_url are production URLs Confirm no TEST keys present in production env. Ensure code chooses keys strictly by environment (no auto fallback). 3 CHECKOUT/UPGRADE FLOW Implement/verify upgrade path: - Authenticated user initiates upgrade - Backend creates Checkout Session (subscription mode) with correct price_id and customer (reuse if exists) - Return session URL to frontend - Frontend redirects - On success, user returns to success page and app shows “upgraded” state AFTER webhook confirms (do not trust query params). Add a “pending activation” UI state until webhook processed. 4 WEBHOOK RELIABILITY (LIVE) Ensure LIVE webhook endpoint is configured in Stripe Dashboard and reachable. Implement idempotency: store event_id unique; ignore duplicates. Handle at minimum: checkout.session.completed, customer.subscription.created/updated/deleted, invoice.payment_succeeded/failed. Update billing_state/plan_tier accordingly. Add alert/log if

webhook fails. 5 CUSTOMER PORTAL Configure Stripe Customer Portal (LIVE) for plan changes/cancel/updates (if used). Ensure portal return URL correct and access controlled. 6 VALIDATION + OBSERVABILITY Add logs/metrics for billing: - checkout session created count - webhook success/failure count - time from checkout completion → entitlement active - mismatch detector (Stripe says active but DB not) Add a reconciliation job or admin sync endpoint to repair drift (minimal). 7 SECURITY Ensure: - Webhook signature verified with LIVE secret - Replay protection with tolerance - Only server creates sessions - Price IDs are allowlisted server-side (prevent user selecting arbitrary price). 8 END-TO-END LIVE TEST PLAN Provide a safe LIVE validation procedure: - Create a €1/€2 monthly “verification” price (temporary) or use existing cheapest plan - Run one full upgrade with a real payment method (authorized) - Verify: Stripe subscription active, DB updated, entitlements enforced, invoice created, portal works - Simulate failure (use cancel before pay) and confirm no entitlement granted - Cancel subscription and confirm entitlement removed per policy. OUTPUT FORMAT Return only: SUMMARY (≤6 bullets) CONFIG (env vars + where set) CODE_CHANGES (file paths + code blocks) STRIPE_DASHBOARD_STEPS (bullets) WEBHOOK_EVENTS (table: event → action) E2E_LIVE_CHECKLIST (steps + expected results) OBSERVABILITY (what to log/measure) ROLLBACK_STEPS No extra commentary.

prompt8:
ROLE Senior billing reliability engineer GOAL Make billing operations and Stripe webhook processing production-grade: idempotent billing-mutating endpoints, webhook event audit table with replay-safe processing, retries + dead-letter handling, and entitlement cache with TTL + webhook-driven invalidation. RULES Stripe is source of truth. No double-charges. No duplicate state transitions. No silent drops. All flows must be deterministic, observable, and testable. TASKS 1 IDENTITY + SURFACE List all billing-mutating endpoints and jobs (create checkout, upgrade/downgrade, cancel, portal, apply coupon, invoice preview, etc.). Identify where Stripe calls are made. 2 IDEMPOTENCY FOR ENDPOINTS Add idempotency keys to every billing-mutating endpoint: - Accept `Idempotency-Key` header from client OR generate stable key server-side per user+operation+params. - Store request fingerprint + result in DB with unique constraint (user_id, operation, idempotency_key). - If same key repeats, return the exact same response without re-calling Stripe. - For Stripe API calls, also pass Stripe’s Idempotency-Key (derived from the same key) to prevent duplicate charges at Stripe. - Define TTL/cleanup for stored idempotency records. 3 WEBHOOK EVENT AUDIT TABLE Create DB table `stripe_webhook_events` (or equivalent): columns: id, event_id (unique), type, created, livemode, payload_json (or minimal), received_at, processed_at, status (received|processed|failed|deadletter), attempt_count, last_error, next_retry_at. Enforce unique(event_id). Always write row first (received) before processing. 4 REPLAY-SAFE PROCESSING Webhook handler logic: - Verify signature + timestamp tolerance. - Upsert event row by event_id; if already processed, return 200 immediately. - Process inside transaction: apply state changes idempotently (use stripe ids + upserts). - Mark processed_at + status=processed only on success. - On failure: status=failed, store last_error, increment attempt_count, schedule retry. 5 RETRIES + DEAD LETTER Implement retry mechanism: - Use existing queue (Celery/Redis) if present; otherwise implement DB-driven retry worker. - Backoff schedule (e.g., 1m, 5m, 15m, 1h, 6h, 24h; max attempts N). - After max attempts, mark

status=deadletter and alert (log + notification hook). - Provide admin endpoint/script to reprocess deadletter events by event_id (secure). 6 ENTITLEMENT CACHE Add entitlement cache with short TTL (e.g., 60–300s): - Cache key: user_id → {billing_state, plan_tier, entitlements, last_synced_at}. - Middleware reads cache first, falls back to DB. - Invalidate cache on webhook events that affect the user (subscription/invoice events). - Also invalidate on billing-mutating endpoint success. - Ensure cache stampede protection (singleflight/lock) if needed. 7 OBSERVABILITY Add metrics/logs: - idempotency_hit_rate per endpoint - webhook received/processed/failed/deadletter counts - webhook processing latency (received → processed) - retry queue depth and deadletter size - entitlement cache hit rate + stale rate. 8 TESTS Add tests for: - endpoint idempotency: same key doesn't create multiple Stripe sessions/subscriptions - Stripe idempotency key passed - webhook duplicate delivery doesn't double-apply - failed webhook enqueues retry and eventually deadletters - cache invalidation occurs on webhook and endpoint success. OUTPUT FORMAT Return only: SUMMARY (≤6 bullets) DB_CHANGES (migrations + table schema) CODE_CHANGES (file paths + code blocks) RETRY_POLICY (schedule + max attempts) CACHE_POLICY (TTL + invalidation triggers) TESTS_ADDED (list) VERIFICATION_STEPS (curl/cli) ROLLBACK_STEPS No extra commentary.

prompt 9:
ROLE Senior product+billing engineer GOAL Make billing UX and ops production-grade: consistent PLAN_REQUIRED errors with upgrade_url+reason codes, past_due/unpaid banners, customer portal with guardrails, invoices (emails + history), EU/FR invoice fields, grace-period rules, and billing observability. RULES Server is source of truth for entitlements. UI must be driven by explicit API fields. Errors must be stable (machine-readable codes). No silent billing failures. All changes must be testable end-to-end. TASKS 1 STANDARDIZE BILLING ERROR ENVELOPE Define a single error schema for authorization/billing failures. For PLAN_REQUIRED always return: - error_code: "PLAN_REQUIRED" - reason_code: stable enum (e.g., FEATURE_LOCKED, LIMIT_EXCEEDED, TRIAL_ENDED, PLAN_MISMATCH) - upgrade_url: stable URL from config (not hardcoded per client) - required_plan: enum - current_plan: enum - request_id Ensure all protected endpoints use the same error builder. Add tests verifying presence of fields. 2 UI BILLING BANNERS Implement app-wide billing banner component driven by /me or /billing/status API: - Show banner when billing_state in {past_due, unpaid, canceled} per policy - Copy: concise state-specific message + CTA - CTA: "Update payment method" (portal) for past_due/unpaid, "Reactivate" (upgrade) for canceled, "Upgrade" for plan_required states - Ensure banner appears on all app pages (layout shell). 3 CUSTOMER PORTAL + GUARDRAILS Add API endpoint to create Stripe customer portal session and return URL. Guardrails: prevent downgrades below active usage constraints: - Define usage constraints (e.g., max sessions, max exports, max workspaces, retention, compute minutes). - Before allowing portal downgrade, compute current usage and block downgrade if it would violate constraints OR enforce "effective at period end" only. - Implement either: (A) server-side downgrade endpoint that validates constraints and then calls Stripe, OR (B) configure portal to disallow plan changes and do upgrades/downgrades via your own validated flow. Choose one and implement. 4 INVOICE RECEIPTS + IN-APP HISTORY Ensure invoices are visible and receipts are sent: - Confirm Stripe customer email is set and receipt emails are enabled (or

implement your own email via webhook invoice.payment_succeeded). - Implement /billing/invoices endpoint that lists invoices (from Stripe or cached DB) with: invoice_id, date, amount, currency, status, pdf_url/hosted_invoice_url. - Add “Billing” page in-app showing invoice history + “Open invoice” links. 5 EU/FR VAT + INVOICE LEGAL FIELDS Ensure invoice data collection + completeness: - Add billing details form (company name, address, country, VAT ID optional, customer type B2C/B2B). - Validate required fields for FR/EU invoicing. - Store in DB and sync to Stripe customer fields and tax IDs. - If using Stripe Tax, configure; otherwise store VAT mode and invoice notes. - Ensure invoices show legal seller info elsewhere (already handled) and customer info completeness. 6 GRACE PERIOD POLICY Implement explicit grace rules to reduce churn while protecting revenue: - Define policy matrix by billing_state and action: ACCESS_VIEW, EXPORT, COMPUTE, UPLOAD, API_WRITE. - Example: past_due: view allowed, compute/export blocked after N days; unpaid: view limited, export/compute blocked; canceled: view allowed for X days then read-only. - Enforce server-side in middleware with clear reason_code (PAST_DUE_GRACE, UNPAID_LOCK, CANCELED_LOCK). - UI must reflect the same policy (banner + disabled buttons + tooltips). 7 BILLING OBSERVABILITY Add metrics/logs + dashboard specs: - webhook_success_rate (processed/received) - time_to_activate (checkout completed → entitlement active) - upgrade_funnel (click upgrade → checkout created → checkout completed → active) - portal_session_created count - payment_failed count and recovery rate - plan_required error rate by endpoint/feature Include request_id and user_id (hashed) for correlation. OUTPUT FORMAT Return only: SUMMARY (≤6 bullets) API_CHANGES (endpoints + schemas) UI_CHANGES (components/pages) BILLING_POLICY (matrix table) STRIPE_CONFIG_STEPS (bullets) DATA_MODEL_CHANGES (billing profile fields) METRICS (names + definitions) TESTS_ADDED (list) MANUAL_VERIFY_STEPS ROLLBACK_STEPS No extra commentary.

prompt 10:
ROLE Senior SRE/platform engineer GOAL Implement production-grade observability + reliability for Lap: uptime monitoring, APM, structured logs with request_id, log aggregation+retention, server log rotation, API access logs with redaction, metrics (Prometheus/StatsD), Grafana dashboards, alerting, SLOs, incident response/runbooks, RTO/RPO+DR, off-site backups+restore tests, Postgres monitoring+pooling, async compute via Celery, queue alarms, circuit breakers, status page, release health, capacity planning, load testing. AUDIT 2026-03-03-full-system-audit. RULES Minimal viable but real. No silent failures. Everything must be verifiable with commands and dashboards. Prefer simple stack choices that fit Hetzner + Cloudflare. Use existing components if present. TASKS 1 INVENTORY Discover current stack: logging (stdout/files), reverse proxy (Caddy), app (Flask/Gunicorn), workers (Celery?), Redis, Postgres, deployment method, existing monitoring (if any). Output minimal inventory (≤10 lines). 2 UPTIME MONITORING Add external synthetic monitoring for: - frontend root - app shell - api /healthz - 1 critical authenticated journey (token-based) - 1 critical compute endpoint (lightweight) Choose provider (Uptime Kuma self-hosted OR BetterStack/Pingdom/etc) and implement minimal setup + alert routing. 3 APM Add tracing + error breakdown: choose OpenTelemetry SDK with exporter (Grafana Cloud, Honeycomb, Sentry Performance, or self-host Tempo). Instrument Flask + requests + DB + Redis + Celery. Ensure trace_id/request_id correlation. 4 STRUCTURED LOGS Switch backend + workers to JSON

logs. Include fields: timestamp, level, service, env, request_id, trace_id, user_id_hash (optional), route, method, status, latency_ms. Ensure request_id propagation across API → worker tasks. 5 ACCESS LOG MIDDLEWARE Add API request logging middleware: log method/path/status/latency/bytes/ip/user_agent; redact secrets (Authorization, cookies, tokens, passwords, Stripe signatures). 6 LOG AGGREGATION + RETENTION Choose one: - Loki+Promtail (self-host) + Grafana - ELK/Opensearch - Cloudflare Logpush + storage + viewer Implement ingestion for: API logs, worker logs, Caddy access logs, Cloudflare edge logs (if available). Set retention policy (e.g., 14–30d hot). 7 LOG ROTATION Configure server log rotation so disk cannot fill (journald limits or logrotate). Provide exact config. Add disk usage alert. 8 METRICS Add Prometheus/StatsD metrics: - HTTP latency histograms by route - HTTP error rates - Celery queue depth + task duration - DB pool usage / connections - pipeline durations (upload→parse→derive→analyze) - system metrics (CPU/RAM/disk) Use exporters: node_exporter, postgres_exporter, redis_exporter; Celery exporter or custom. 9 DASHBOARDS Create Grafana dashboards: - SLO overview (availability/latency) - API (p95/p99 latency, 4xx/5xx, top routes) - Workers (queue depth, task duration, failures) - Postgres (connections, slow queries, locks) - Redis (memory, evictions) - Infra (CPU/RAM/disk) - Billing/webhooks (processed/failed latency) - Release health (error rate by version/build). 10 ALERTING Configure alerts: - availability failing (synthetics) - 5xx spike - p95 latency spike - queue backlog - task failure spike - DB connections high / locks / replication (if any) - disk >80% - CPU/RAM sustained high - backup failures - webhook failures. Define routing (email/Slack/Discord) + oncall policy. 11 SLOS Define SLOs for journeys: login, upload, analysis, compare, export. For each: target availability + latency threshold + measurement method + error budget. 12 INCIDENT MGMT Produce: severity matrix (SEV0-3), incident comms templates (internal + customer), runbooks for: DB down, Redis down, worker backlog, Stripe webhook failures. 13 RTO/RPO + DR Set RTO/RPO for DB and object storage. Document recovery steps. Write DR plan: restore order, dependencies, verification checklist. 14 BACKUPS Off-site backups (S3-compatible). Automate: Postgres dumps/basebackup + app config secrets (safe) + object storage metadata. Encrypt backups. Implement monthly automated restore test in staging and record evidence. 15 POSTGRES PERF Enable pg_stat_statements, expose metrics, and add top query dashboard. Add PgBouncer (transaction pooling) and adjust app DSN. 16 ASYNC CPU Ensure long CPU tasks not blocking Gunicorn: move to Celery (or confirm already). Provide refactor steps: endpoints enqueue tasks, return job id, poll/result endpoints. 17 CELERY ALARMS Add queue depth + task duration alarms + failure alarms. 18 CIRCUIT BREAKERS Add dependency timeouts + circuit breakers: DB/Redis/third-party. On degrade return fast 503 with stable error_code and request_id. 19 STATUS PAGE Add basic public status page + incident history (static or hosted). 20 RELEASE HEALTH Add build/version stamp to frontend+api responses and logs. Create dashboard: error rate by version. 21 CAPACITY PLAN Create a short capacity doc: current VPS specs, expected concurrent users, max concurrent analyses, scaling triggers, next scaling steps. 22 LOAD TESTS Add load tests for upload+analysis: k6/locust scripts, define target concurrency, record results, set scaling thresholds. OUTPUT FORMAT Return only: SUMMARY (≤8 bullets) STACK_CHOICES (what tools selected) CONFIG_CHANGES (files + code blocks) DASHBOARDS (list + panels) ALERTS (list + thresholds) SLOS (table) RUNBOOKS (bullets per incident) BACKUP_PLAN (steps + schedule) RESTORE_TEST (monthly procedure) LOAD_TEST_SCRIPTS (paths +

commands) VERIFICATION (commands/checks) ROLLBACK_STEPS No extra commentary.

prompt 11:ROLE Senior platform engineer GOAL Deliver production-grade delivery pipeline: staging env with prod topology, strict env parity, CI/CD for API+frontend, PR protections, preview deploys, migration safety checks, tests (unit/integration/E2E/contract), perf budgets, pre-commit tests, post-deploy smoke tests, rollback strategy, IaC, non-root deploy, bastion/VPN, intrusion detection, and a release process. AUDIT 2026-03-03-full-system-audit. RULES Minimal but real. No manual “cowboy” deploys. No direct pushes to main. All changes verifiable. Prefer Cloudflare-native for frontend and simple reproducible Linux tooling for API. TASKS 1 STAGING TOPOLOGY Create staging environment mirroring prod topology: Cloudflare → Caddy → API app + workers + Redis + Postgres. Use separate domain(s): staging.*. Ensure isolated DB/Redis, separate Stripe keys (test), separate secrets. 2 ENV PARITY Implement config scheme shared across envs: - single config loader - .env.example with all vars - strict validation: required vars per env; fail fast - consistent naming: APP_ENV=dev|staging|prod - eliminate env-specific code branches except config values. 3 REPO GOVERNANCE Enforce branch protections: - require PR review - required status checks - disallow direct pushes to main - require signed commits (optional) - CODEOWNERS for critical paths (infra/billing/security). 4 FRONTEND CI/CD Use Cloudflare Pages CI: - PR preview deployments enabled - build + lint + typecheck + tests - deploy on merge to main to production - staging deploy from develop/release branch OR separate Pages project. 5 API CI/CD Build full pipeline: - build artifact (Docker or tarball) - run backend tests - run migration checks - deploy to staging automatically - run post-deploy smoke tests - promote to production (manual approval gate or tag-based) - rollback supported. 6 MIGRATION SAFETY Add CI checks for Alembic: - revision consistency - upgrade head on fresh DB - downgrade one step (where safe) OR expand/contract check - block non-reversible migrations unless explicitly approved. 7 BACKEND TESTS Add unit + integration tests for critical endpoints and RBAC variants. Include fixtures for: free/paid/admin roles, rate limits, billing errors, upload/analyze/export. 8 FRONTEND TESTS Add CI workflow: - smoke tests (build + render key routes) - component tests - minimal E2E (Playwright preferred). 9 E2E COVERAGE Implement Playwright/Cypress E2E for: register/login, upload, analyze, compare, billing upgrade, export. Must run against staging (or ephemeral env) with seeded test data. 10 CONTRACT TESTS Define and enforce API response envelope contract: - success schema - error schema (code, message, request_id, reason_code, upgrade_url where needed) Add tests to ensure consistency across endpoints. 11 PERF BUDGET Add CI checks: - bundle size thresholds per route - LCP budget for key route(s) via Lighthouse CI (or similar). Fail CI if exceeded. 12 PRE-COMMIT Upgrade pre-commit to run: lint + typecheck + unit tests (fast subset). 13 POST-DEPLOY SMOKE Add automated smoke tests after deploy: - /healthz - auth check - 1 core read endpoint - 1 core write endpoint (staging only) Must gate promotion. 14 ROLLBACK STRATEGY Implement versioned deploys + rollback: - artifact versioning (git sha/tag) - blue/green or canary (simple) - DB migration strategy: expand/contract, feature flags, no destructive changes without two-phase rollout. 15 IaC Add Terraform/Ansible for servers: - provisioning, users, firewall, Caddy, services, env vars, backups, monitoring agents. Must recreate staging/prod reliably. 16 DEPLOY USER Create non-root deploy user with least privilege; systemd services run as non-root. 17 BASTION/VPN Add admin access hardening: - Tailscale/WireGuard or bastion host - restrict SSH to

VPN/bastion - optionally require MFA at identity layer. 18 INTRUSION DETECTION Add Wazuh/OSSEC (or equivalent) with alerts and basic rules. 19 RELEASE PROCESS Define release policy: - semantic versioning or date+sha - changelog generation - stop-the-line criteria (SLO regression, security fail, failing tests) - release checklist and approvals. OUTPUT FORMAT Return only: SUMMARY (≤8 bullets) ENV_SETUP (staging domains + topology + env vars) CI_WORKFLOWS (files/paths + steps) CLOUDFLARE_SETUP (preview/staging/prod) API_DEPLOY (artifact + deploy + rollback) MIGRATION_POLICY (rules + CI gates) TEST_PLAN (backend/frontend/e2e/contract) PERF_BUDGETS (thresholds) IaC_PLAN (tools + structure) ACCESS_HARDENING (deploy user + VPN/bastion) IDS_SETUP (tool + alerts) RELEASE_PROCESS (checklist) VERIFICATION_STEPS ROLLBACK_STEPS No extra commentary.

prompt 13: ROLE Senior telemetry/data engineer GOAL Fix bug where speed trace endpoint returns empty `traces: []` despite available data; ensure production-grade correctness, observability, and regression coverage. AUDIT 2026-03-03-full-system-audit: “speed trace returns empty traces array” while data exists. RULES No silent failures. If trace cannot be produced, return explicit error_code + diagnostics (not empty success). Preserve backward compatibility if possible. Must include reproducible test + fix. TASKS 1 REPRODUCE Locate API endpoint that returns speed traces (and any service/pipeline function it calls). Create a minimal reproduction using: - a known session/lap in DB (or fixture file) that has speed data - direct call to endpoint returning empty traces. Capture inputs, response, and server logs. 2 TRACE THE DATA PATH Identify full data path: upload → parsing → derived channels → storage → query → response serialization. Determine where data becomes empty: - parsing yields no speed? - derived channels missing? - query uses wrong column/unit? - filtering drops all points? - lap window slicing wrong? - interpolation resample bug? - serialization expects a different key? Add temporary instrumentation to confirm point counts at each step. 3 FIX ROOT CAUSE Implement fix at correct layer (prefer pipeline/query logic, not UI hacks). Ensure: - speed channel existence detection robust (support synonyms: speed, v, gps_speed, etc.) - unit normalization consistent - time/distance axis alignment correct - filtering cannot drop all points silently (guardrails) - if lap slicing uses indices, validate bounds and ordering. 4 FAIL LOUD (NO EMPTY SUCCESS) If trace generation fails or yields 0 points, return: - HTTP 422 (or 500 if internal) - error_code: SPEED_TRACE_EMPTY - reason_code: NO_SPEED_CHANNEL | ALL_POINTS_FILTERED | INVALID_LAP_WINDOW | STORAGE_MISSING | OTHER - request_id - debug_fields (counts, selected channel name, filters applied) gated by env (debug only). 5 REGRESSION TESTS Add tests: - unit test: given sample telemetry with speed, trace array non-empty and monotonic axis - integration test: endpoint returns traces length > 0 for fixture session - negative tests: missing speed channel returns SPEED_TRACE_EMPTY with reason_code. Add golden fixture(s) to repo or generate synthetic telemetry deterministically. 6 OBSERVABILITY Add metrics/logs: - speed_trace_requests - empty_trace_failures by reason_code - median trace point count - pipeline step duration for trace build. Ensure logs include selected channel + counts (no PII). 7 VALIDATION Provide curl steps to validate fix on staging: - endpoint returns non-empty traces - error cases return explicit error_code - no empty success responses remain. OUTPUT FORMAT Return only: SUMMARY (≤6 bullets)

ROOT_CAUSE (≤3 bullets) CODE_CHANGES (file paths + code blocks) TESTS_ADDED (list) METRICS_LOGS (what added) CURL_VERIFY (commands + expected) ROLLBACK_STEPS No extra commentary.

prompt 14: ROLE Senior data/pipeline engineer GOAL Make analysis pipeline trustworthy: include per-step status in outputs, surface safe-step warnings as user-visible “analysis degraded” flags with step names, and add deterministic pipeline+model versioning stored with results. AUDIT 2026-03-03-full-system-audit: silent failures via _safe_step and missing pipeline versioning. RULES No silent failures. Success responses must include step statuses. Degraded analysis must be explicit. Versioning must be deterministic and reproducible. Backward compatible API where possible. TASKS 1 INVENTORY Locate pipeline orchestration code (step runner, _safe_step, analysis result builder, DB storage). List all pipeline steps in order. Identify where warnings/errors are currently swallowed or only logged. 2 DEFINE STEP STATUS SCHEMA Add a minimal schema included in every analysis response: pipeline: { pipeline_version, model_version, status: ok|degraded|failed, degraded: bool, degraded_steps: [step_name], steps: [{name, status: ok|warn|fail|skipped, started_at, duration_ms, message_code, message, metrics:{...}}] } Keep message fields short; include machine-readable message_code. Ensure schema is stable and documented. 3 IMPLEMENT STEP CAPTURE Modify step runner to capture for each step: start/end time, status, warning/error codes, and key metrics (rows/points in/out). Ensure _safe_step returning warnings sets status=warn, not ok. If a step fails but pipeline continues, mark warn/fail accordingly and set pipeline.status=degraded. If pipeline cannot proceed, set pipeline.status=failed and return explicit error with step context. 4 USER-VISIBLE “ANALYSIS DEGRADED” When any step is warn/fail-but-continued: - set degraded=true - include degraded_steps list - include a short user_message string like “Analysis degraded: <step1>, <step2>” Ensure UI can display this without deep parsing. 5 STORAGE + VERSIONING Add deterministic versioning persisted with results: - pipeline_version: derived from code version (git sha/tag) + pipeline definition hash (hash of ordered step names + step parameter schema). - model_version: derived from model artifact identifier (registry version, checksum) or “none” if not used. Store both in DB table for analysis results and include in API responses. Ensure version values are identical for identical code/model builds. 6 BACKWARD COMPAT Add fields without breaking existing clients: keep existing response keys, add new `pipeline` object. If existing `warnings` exists, map it to pipeline.steps statuses. 7 TESTS Add tests: - degraded when _safe_step warns, includes degraded_steps - step status array includes all steps with correct statuses - pipeline_version stable across runs (same code) - model_version stored and returned - failure returns includes failing step name + request_id. 8 MIGRATION Add DB migration to store pipeline_version and model_version on analysis results. Provide backfill strategy (set unknown for existing). 9 VERIFICATION Provide curl examples showing: - normal ok pipeline response includes steps[] - degraded response includes degraded=true + step names - stored versions returned consistently. OUTPUT FORMAT Return only: SUMMARY (≤6 bullets) API_SCHEMA (minimal JSON example) CODE_CHANGES (file paths + code blocks) DB_MIGRATION (if any) VERSIONING_RULES (how computed) TESTS_ADDED (list) CURL_VERIFY (commands + expected fields) ROLLBACK_STEPS No extra commentary.

prompt 15: ROLE Senior telemetry QA/data engineer GOAL Increase trust and prevent regressions by adding golden datasets + regression tests for derived channels/outputs, unit tests for core domain metrics (lap segmentation, distance axis, delta, time loss attribution), configurable outlier filtering by track/session type, and UI overrides for vehicle/kart parameters (mass, power). AUDIT 2026-03-03-full-system-audit. RULES Tests must be deterministic and run in CI. Golden datasets must be versioned, small, and privacy-safe. Config must be explicit and environment-safe (no hidden magic). UI overrides must be validated and reflected in outputs + metadata. TASKS 1 GOLDEN DATASETS Create a “golden” dataset suite: - 3–10 sessions max, representative (clean GPS, noisy GPS, different tracks, different kart types, different lap counts). - Store in repo as small fixtures (compressed CSV/Parquet) or generate from embedded seed + generator. - Remove/obfuscate personal data. - Provide a manifest (track, expected laps, expected key metrics ranges). 2 REGRESSION TESTS (DERIVED + OUTPUTS) Add regression tests that run pipeline on golden sessions and assert stable outputs within tolerances: - derived channels exist (speed, accel, yaw rate, distance, lap_id, etc.) - output objects non-empty (speed trace, racing line points, delta arrays) - key summary metrics within tolerance bands (not exact equality). Save expected snapshots as JSON with tolerances per field. 3 CORE DOMAIN UNIT TESTS Implement focused unit tests for: - lap segmentation correctness: detects lap boundaries, lap count, lap durations monotonic. - distance axis: monotonic increasing, correct total length within tolerance, consistent sampling. - delta computation: sign conventions, alignment, stable endpoints. - time loss attribution: per-corner loss sums approximate total loss; no negative nonsense; corner boundaries consistent. Include synthetic telemetry generators for deterministic tests (simple circle track, known speed profile). 4 CONFIGURABLE OUTLIER FILTERING Replace hardcoded thresholds with configuration: - config keys: gps_jump_m, speed_spike_mps, accel_spike_mps2, heading_jump_deg, min_samples_per_lap, etc. - config selection based on track_id + session_type + sensor_profile. - default config plus per-track overrides (YAML/JSON). - store chosen config version/name in analysis result metadata. Add tests ensuring override applied and logged. 5 VEHICLE/KART PARAM OVERRIDES Add UI + API support for overrides: mass_kg, power_kw (or hp), drag model selector (optional). Flow: - UI exposes overrides (advanced settings) with defaults from vehicle preset - Validate ranges server-side - Persist overrides on session or analysis request - Pipeline uses overrides in model computations - Include overrides + source (default vs user) in outputs metadata. 6 CI INTEGRATION Ensure tests run in CI within time budget: - mark golden tests as “regression” group - allow subset on PR, full on main nightly if needed. Provide commands. OUTPUT FORMAT Return only: SUMMARY (≤6 bullets) GOLDEN_DATASET_PLAN (files + manifest fields) TESTS_ADDED (list by type) TOLERANCE_RULES (how comparisons done) CONFIG_SYSTEM (file format + selection logic) UI_API_CHANGES (endpoints + fields) METADATA_ADDED (what stored/returned) CI_WIRING (commands/workflows) ROLLBACK_STEPS No extra commentary.

prompt 16: ROLE Senior telemetry platform engineer GOAL Improve analysis performance + trust: intermediate caching, idempotent analyses, time budgets + cancellation, upload integrity checksums, units/sampling validation with explicit reporting, and GPS/IMU sync validation with a confidence score shown in UI. AUDIT 2026-03-03-full-system-audit. RULES No silent failures. Deterministic results. Cache must be safe (no cross-tenant leakage). Cancellation must stop

CPU work. Every auto-fix must be reported. TASKS 1 INTERMEDIATE CACHING Design and implement step-level caching so expensive steps aren't recomputed if inputs unchanged. Requirements: - Cache key includes: workspace_id, session_id, pipeline_version, step_name, step_params_hash, input_artifact_hash. - Store artifacts (e.g., parsed canonical telemetry, derived channels, resampled series, lap segmentation, track map) as Parquet/Arrow/NPZ + metadata JSON. - Cache is immutable; write-once, read-many. - Cache storage: local disk + content-addressed paths OR object storage; pick one. - Add cache hit/miss metrics + logs. - Add TTL/cleanup policy and size cap. 2 IDEMPOTENT ANALYSIS Implement idempotent analysis keyed by: (workspace_id, session_id, pipeline_version, params_hash). - Before enqueueing analysis, check DB for existing result with same key and status=completed; return it. - If status=running, return existing job_id. - Ensure unique constraint on (workspace_id, session_id, pipeline_version, params_hash). - params_hash must be deterministic (sorted JSON canonicalization). 3 TIME BUDGET + CANCELLATION Implement analysis time budget + cancel: - Add max_runtime_seconds per analysis (configurable by plan). - Worker enforces soft timeout (graceful stop) + hard kill (if supported). - Provide API endpoint: POST /analysis/{id}/cancel. - Cancellation propagates to Celery task: check cancellation flag between steps and inside long loops; stop quickly and mark status=canceled. - Ensure partial artifacts are either discarded or marked incomplete (no reuse unless safe). 4 UPLOAD INTEGRITY CHECKSUMS On upload: - Compute checksum (SHA-256) of raw file bytes and store. - Verify checksum after storage write/read; reject if mismatch. - Include checksum in session metadata and in exports. - Add duplicate upload detection (same checksum within workspace) optional. 5 UNITS + SAMPLING VALIDATION Add validation step pre-pipeline: - Detect units (speed m/s vs km/h, accel g vs m/s², gyro deg/s vs rad/s) using heuristics + metadata; allow explicit user override. - Validate sampling rates for GPS and IMU; detect irregular sampling, missing timestamps, time going backwards. - Decide per issue: reject, auto-fix, or warn. - Report outcomes in pipeline step status: message_code + details (what fixed, what assumed). 6 GPS/IMU SYNC VALIDATION + CONFIDENCE SCORE Implement robust sync check: - Estimate time offset between GPS and IMU (cross-correlation of yaw rate vs heading rate / lateral accel vs curvature). - Output: offset_ms estimate, jitter_ms, confidence_score 0–1, and labels (GOOD/OK/POOR). - If confidence low, mark analysis degraded and surface UI banner. - Store sync metrics in results metadata. 7 UI SURFACING Add UI surfaces for: - cache hit indicator optional (debug) - analysis status: running/completed/canceled/timeout - validation report (units/sampling) - sync confidence score + tooltip with offset/jitter. OUTPUT FORMAT Return only: SUMMARY (≤7 bullets) DATA_MODEL_CHANGES (tables/fields) CACHE_DESIGN (keys + artifact list + storage paths) IDEMPOTENCY_RULES (unique keys + flow) CANCELLATION_FLOW (API + worker checks) VALIDATION_RULES (issues → reject/fix/warn) SYNC_METHOD (inputs + outputs) UI_CHANGES (screens/components) METRICS (names) TESTS_ADDED (list) VERIFICATION_STEPS (curl/cli) ROLLBACK_STEPS No extra commentary.

prompt 17: ROLE Senior telemetry/ML engineer GOAL Add trust features: track map validation + wrong-track detection that blocks coaching, per-corner sensitivity/uncertainty surfaces (confidence heatmap), explainability bundle for each insight, and export reproducibility with computed-at metadata. RULES No misleading coaching. Deterministic outputs for same inputs.

Explanations must be concise, structured, and machine-readable. UI must surface confidence clearly. TASKS 1 TRACK MAP VALIDATION + WRONG-TRACK DETECTION Implement track validation before generating coaching: - Inputs: GPS path (lat/lon), derived curvature/heading, lap segmentation. - Compare session to known track library (centerline/track polygon) using: start/finish proximity, lap length, shape similarity (DTW/Frechet), and alignment score. - Output: detected_track_id (or unknown), match_score 0–1, and mismatch_reason codes. - If match_score below threshold or track unknown: block coaching insights and show “Track not confirmed” message; allow raw telemetry viewing. - Add override flow: user can manually select track to proceed (logged) OR require confirmation step. 2 PER-CORNER SENSITIVITY / UNCERTAINTY SURFACES Build confidence heatmaps per corner: - Define corner segments (from track map or curvature segmentation). - For each corner compute uncertainty score based on: GPS noise, sync confidence, speed signal quality, sample density, lap-to-lap variance, model residuals. - Optionally run local perturbation around key decisions (brake point/apex/throttle) to estimate sensitivity of lap time (Δt per Δm) within bounded range. - Output: per-corner confidence 0–1 + sensitivity metrics (e.g., ms per meter brake/apex shift) + flags. - UI: show heatmap overlay on track + corner list sorted by low confidence/high sensitivity. 3 EXPLAINABILITY BUNDLE For every coaching insight produce an “explainability bundle”: - insight_id, type, severity, recommendation - evidence: list of signals with contributions (e.g., throttle_onset_delta_ms, brake_release_delta_m, min_speed_delta, lateral_g_utilization, line_deviation_m) - context: corner_id, lap_id(s), baseline/reference lap, conditions flags (traffic/noise) - confidence: numeric + reasons - links: pointers to trace segments used (time/distance ranges) Keep bundle small and stable schema; no free-form walls of text. 4 EXPORT REPRODUCIBILITY + METADATA Ensure same inputs yield same outputs: - Canonicalize inputs: normalized units, sorted params JSON, stable resampling/interpolation seeds, deterministic floating rounding policy. - Record in every result/export: computed_at (UTC), pipeline_version, model_version, params_hash, input_checksums (raw upload sha256 + derived artifact hashes), detected_track_id, sync offset/confidence. - Export files include a manifest.json with all metadata and schema version. - Add verification utility/endpoint: given manifest, confirm reproducibility by recomputing and comparing hashes within tolerance. 5 API + UI WIRING Add API fields for: track_validation, corner_confidence, sensitivity, insights[].explainability. UI: - “Track confirmed” badge or blocking warning - confidence heatmap toggle - insight details drawer with “Why?” section populated from explainability bundle - export panel showing computed_at + versions + params_hash. 6 TESTS Add tests for: - wrong-track session triggers block - known track matches above threshold - confidence heatmap computed and stable - explainability bundle schema validation - reproducibility: same fixture inputs produce identical manifest hashes. OUTPUT FORMAT Return only: SUMMARY (≤7 bullets) API_SCHEMA (minimal JSON shapes) TRACK_VALIDATION (scoring + thresholds + block rules) CONFIDENCE_HEATMAP (how computed) EXPLAINABILITY_SCHEMA (fields + examples) REPRODUCIBILITY_MANIFEST (fields) UI_CHANGES (components) TESTS_ADDED (list) VERIFICATION_STEPS (curl/cli) ROLLBACK_STEPS No extra commentary.

prompt18: ROLE Senior privacy+data engineer GOAL Enforce 12-month telemetry retention with explicit deletion jobs and audit logs, and implement privacy-safe pseudonymization for

session identifiers in logs/exports. RULES GDPR-friendly. Deletions must be irreversible (or clearly defined soft→hard). No silent failures. Audit logs must be tamper-evident and retention-controlled. Pseudonyms must be stable per workspace (or globally) but non-reversible without a secret. TASKS 1 INVENTORY Identify all telemetry storage locations and identifiers: DB tables (sessions, laps, analysis results), file/object storage (uploads, derived artifacts, exports, caches), logs, backups. Identify which fields are personal data. 2 RETENTION POLICY Implement explicit retention policy: - Default retention: 12 months from session created_at (or last_activity_at; choose and document). - Define what is deleted: raw uploads, derived artifacts, analysis outputs, exports, cached intermediates, associated DB rows. - Define exceptions: active legal hold/admin override (optional) and how it's tracked. 3 ENFORCEMENT JOBS Implement scheduled jobs (cron/Celery beat): - daily "identify-expired" job that lists deletions due - "delete-expired" job that performs deletion in correct order: revoke access → delete files → delete DB rows (or soft delete then hard delete) - ensure idempotent (safe reruns) - backpressure controls (batch size, rate limit). 4 AUDIT LOGGING Create an audit log system for retention actions: - table `retention_audit` with: id, run_id, workspace_id, subject_type (session/export/artifact), subject_id, action (mark|delete|error), reason (retention_expired|user_request|admin), performed_at, actor (system/user/admin), result (success/fail), details (counts, paths), request_id. - keep audit logs for defined period (e.g., 24 months) and document. - include summary per run (counts, duration, failures). 5 PSEUDONYMIZATION Implement privacy-safe pseudonyms for session identifiers in logs/exports: - Define pseudonym id: session_pid = HMAC-SHA256(secret, workspace_id + ":" + session_id) truncated (e.g., 16–24 chars). - secret stored as LOG_PSEUDONYM_SECRET and rotated with dual-secret support. - Use session_pid in all logs and exported manifests instead of raw session_id. - Exports: include session_pid, not internal ids; keep internal ids only inside system DB. - Ensure pseudonyms are stable and collision-resistant; never reversible without secret. 6 ROTATION SUPPORT Add dual-secret rotation: accept current+previous secret to validate/lookup existing pids for a grace period; regenerate new pids for new exports. Document rotation steps. 7 BACKFILL Update logging to emit pid for existing sessions; update export generator to include pid; update any tooling reading exports accordingly. 8 TESTS Add tests for: - retention job deletes all artifacts and DB rows for expired sessions - idempotency (rerun doesn't error) - audit logs created with correct counts - pseudonym stable for same input and changes when secret changes - pseudonym never equals raw id and is length/charset constrained. 9 VERIFICATION Provide commands/queries to: - create session with old created_at, run job, confirm deletion - confirm audit records - confirm logs/export manifests contain only session_pid. OUTPUT FORMAT Return only: SUMMARY (≤6 bullets) DATA_MODEL_CHANGES (tables/fields) RETENTION_JOB (schedule + batch rules) DELETE_ORDER (steps) AUDIT_LOG_SCHEMA (fields) PSEUDONYM_SPEC (algo + formats + rotation) CODE_CHANGES (file paths + code blocks) TESTS_ADDED (list) VERIFICATION_STEPS (sql/curl/cli) ROLLBACK_STEPS No extra commentary.

prompt 19: ROLE Senior frontend engineer GOAL Improve production frontend quality: remove debug console logs in production, fix Chart.js memory leaks by destroying instances on unmount, and ensure every API error shown in UI includes request_id for support. AUDIT 2026-03-03-full-system-audit. RULES No console noise in production. No memory leaks on

route changes. Error UX must be consistent and minimal. Must be covered by tests/lints. TASKS 1 REMOVE DEBUG LOGS Find all console.* usage in frontend (and any debug logger). Remove or gate logs behind DEV-only flag. Implement a minimal logger wrapper: - in PROD: no-op for debug/info, allow error/warn only if needed - in DEV: pass-through. Add ESLint rule (no-console) with explicit allowlist if needed. Ensure build strips dead code if using flags. 2 FIX CHART MEMORY LEAKS Identify Chart.js usage (or wrapper). Ensure every chart instance is destroyed on component unmount and before re-creating on data changes. If using React refs: keep chart instance ref and call chart.destroy() in cleanup. Ensure event listeners and ResizeObserver are disconnected. Add regression check: navigate between pages repeatedly; no growing listeners/heap. 3 REQUEST_ID IN ERROR UI Ensure API returns request_id in error envelope and headers. Frontend must extract request_id from: - JSON body field request_id (preferred) OR - response header X-Request-Id (fallback). Update API client wrapper to always attach request_id to thrown errors. Update all error UI components (toasts, banners, error pages) to show “Request ID: <id>” with copy button. Ensure request_id shown for both HTTP errors and network errors (use “unknown” if missing). 4 TESTS Add: - lint rule enforcement for console.* in production code - unit test for API client error object includes request_id - component test that error toast/banner renders request_id when present. OUTPUT FORMAT Return only: SUMMARY (≤5 bullets) CODE_CHANGES (file paths + code blocks) LINT_RULES (config snippet) TESTS_ADDED (list) MANUAL_VERIFY_STEPS (bullets) ROLLBACK_STEPS No extra commentary.

prompt 20: ROLE Senior frontend engineer GOAL Bring frontend to production-grade UX/perf/reliability: handle silent refresh failures cleanly, improve bundle splitting, mobile LCP, CLS, enforce perf budgets + Web Vitals tracking, add feature flags, reach Lighthouse accessibility >90, add Storybook, enforce TS strictness, and add offline/poor-network resilience for viewing past sessions. AUDIT 2026-03-03-full-system-audit. RULES No user stuck states. Performance changes measurable per release. Feature changes behind flags. Accessibility enforced in CI. Minimize scope but ship real improvements. TASKS 1 AUTH: SILENT REFRESH FAILURE UX Locate auth token refresh flow. Implement: - If refresh fails (401/invalid/timeout), immediately: clear auth state, show modal/toast “Session expired”, and redirect to login with returnTo. - Prevent infinite refresh loops; cap retries; exponential backoff. - Provide “Retry” once + “Log in” CTA. - Ensure all API calls fail fast with consistent error type. 2 ROUTE-LEVEL CODE SPLITTING Implement route-level splitting (Next.js dynamic imports / React.lazy) for heavy pages: session analysis, charts, compare, billing. Ensure common shell stays small. Verify bundle analyzer outputs improve. 3 MOBILE LCP Improve LCP for marketing + app landing: - optimize hero image: responsive sizes, modern formats, priority preload - reduce render-blocking fonts: self-host, subset, preload, font-display swap - avoid heavy JS before first paint (defer non-critical). 4 CLS FIX Reserve chart/container dimensions on session page: - set fixed aspect ratio or min-height placeholders for charts and map - use skeletons matching final size - ensure late-loaded fonts/images don’t shift layout. 5 WEB VITALS + PERF BUDGETS Add: - Web Vitals collection per release (LCP/CLS/INP) with build/version tag - CI perf budgets: bundle size limits + Lighthouse CI for key routes; fail if regressions exceed thresholds - store results per commit/tag. 6 FEATURE FLAGS Add feature flag system: -

server-provided flags or remote config (simple JSON) with caching - client hook to gate features - include kill-switch for risky features - ensure flags logged with version and request_id. 7 ACCESSIBILITY >90 Add fixes and enforcement: - run axe/lighthouse in CI for key pages - fix common issues: labels, focus states, contrast, keyboard nav, heading order, aria for charts/modals - ensure reduced-motion respected. 8 STORYBOOK Add Storybook (or equivalent) for shared components: - docs for buttons/forms/charts/skeletons/banners - visual regression optional - publish via CI artifact. 9 TYPESCRIPT STRICTNESS Enforce TS strict mode: - enable strict, noImplicitAny, strictNullChecks - remove suppressions; add proper types - CI fails on TS errors. 10 OFFLINE/POOR-NETWORK RESILIENCE Add minimal resilience for viewing past sessions: - cache last N session summaries + last opened session data in IndexedDB/localforage - show “offline mode” banner when network down - allow read-only viewing of cached sessions - optional PWA service worker for static assets only. 11 TESTS Add coverage: - refresh failure triggers logout+redirect - route splitting doesn’t break navigation - CLS guard on session page (basic) - feature flag gates work - accessibility smoke test passes. OUTPUT FORMAT Return only: SUMMARY (≤8 bullets) CODE_CHANGES (file paths + code blocks) PERF_BUDGETS (numbers + CI config) WEB_VITALS (how collected + where stored) FEATURE_FLAGS (design + kill switch) A11Y_FIXES (list) STORYBOOK_SETUP (scripts/config) TS_CONFIG (tsconfig changes) OFFLINE_CACHE (what cached + limits) TESTS_ADDED (list) MANUAL_VERIFY_STEPS ROLLBACK_STEPS No extra commentary.

prompt 21: ROLE Senior frontend engineer GOAL Improve perceived performance and reduce layout shift: add route/data prefetching on hover/proximity for key routes and implement consistent skeletons sized to final layout. AUDIT 2026-03-03-full-system-audit. RULES Prefetch must be bounded (no bandwidth explosions). Respect Save-Data, slow connections, and reduced-motion if relevant. Skeletons must match final dimensions to avoid CLS. TASKS 1 KEY ROUTES Identify top navigation routes and high-intent links (e.g., Sessions list → Session detail, Compare, Billing, Export). Define a short allowlist for prefetch targets (max 5–8). 2 ROUTE PREFETCH Implement prefetch on: - hover/focus of links - proximity via IntersectionObserver (when link enters viewport) Use framework-native prefetch where possible (Next.js router.prefetch / <Link prefetch>). Add guardrails: disable prefetch when: - navigator.connection.saveData true - effectiveType in {2g, slow-2g} - user not authenticated (for authed routes) - already prefetched recently (TTL cache). 3 DATA PREFETCH For session detail route: prefetch minimal API data (session summary + traces metadata) into client cache on hover/proximity, using an in-memory cache with TTL. Ensure request_id captured and errors ignored silently (no user-visible errors during prefetch). 4 SKELETON SYSTEM Create a skeleton component library that matches final layout: - skeleton blocks with exact heights/widths/aspect ratios for charts, tables, headers, side panels - use same grid/spacing as final components - ensure skeletons render immediately and are replaced without shifting container size. Apply to key pages: session detail, compare, billing, sessions list. 5 CLS VALIDATION Add a simple CLS guard: - reserve chart container min-height or aspect-ratio CSS - ensure images have width/height or aspect ratio - ensure fonts are preloaded to avoid layout shifts (only if impacted here). 6 TESTS Add: - unit/component tests: skeleton container dimensions stable - navigation prefetch invoked only for allowlisted routes - prefetch disabled on

Save-Data/slow network. OUTPUT FORMAT Return only: SUMMARY (≤5 bullets) PREFETCH_RULES (allowlist + guardrails) CODE_CHANGES (file paths + code blocks) SKELETON_SPECS (per page sizes) TESTS_ADDED (list) MANUAL_VERIFY_STEPS ROLLBACK_STEPS No extra commentary.

prompt 22: ROLE Senior product+privacy engineer GOAL Ship compliance/trust essentials fast: analytics gated by explicit consent (ePrivacy/GDPR), consent UI shows cookie lifetimes, add company physical address to footer, expand sitemap, and add Security page + basic DPA for B2B. Also align work with the fast-path phases (0–3). AUDIT 2026-03-03-full-system-audit. RULES No analytics loads before consent. Consent must be revocable. Keep implementation minimal but correct for EU/FR. All pages must be linked and included in sitemap. TASKS 1 CONSENT-GATED ANALYTICS Identify analytics tooling used (GA, Plausible, PostHog, etc.) and where scripts are loaded. Implement a consent manager: - Default state: no analytics - UI banner with Accept / Reject / Customize - Store consent in first-party cookie/localStorage with versioning - Provide “Change preferences” link in footer. Gate analytics by consent: do not inject script or send events until consent=accepted. Ensure consent is required for non-essential cookies per ePrivacy. 2 COOKIE LIFETIMES IN UI Build a cookie details view listing categories with explicit expiry/lifetime and purpose: - Essential (session/auth) with expiry - Analytics with expiry - Any marketing (if none, state none) Show: cookie name, provider, purpose, expiry, type. Keep it short but present. 3 FOOTER LEGAL INFO Add company physical address in site footer (all public pages) and ensure consistent across marketing + app shell: - Lap Coach - 5 rue Paré, 44000 Nantes, France Include contact email if desired. 4 SITEMAP EXPANSION Expand sitemap to include at least: /, /product, /pricing, /privacy, /terms, /security, /dpa, /contact, /demo (or request demo), plus any key marketing pages. Ensure canonical URLs and robots rules correct. If app is blocked from indexing, exclude /app routes. Add automated generation if using Next.js. 5 SECURITY PAGE Create /security page (v1) covering: - high-level security posture (TLS, encryption, access control) - data retention (12 months) - incident response contact - sub-processors list pointer - vulnerability reporting channel. Keep factual and non-overpromising. 6 DPA BASICS Create /dpa page (v1) for B2B trust: - roles (controller/processor) - data categories - processing purpose - security measures summary - sub-processors - retention - breach notification process - SCC mention placeholder if needed. Keep short and link to request full signed DPA via email. 7 FAST-PATH ALIGNMENT Ensure these changes fit phases: - Phase 0: minimal footer address + consent gating scaffold - Phase 1: publish /security + /dpa + sitemap expansion - Phase 2+: add CI checks for consent gating + sitemap completeness. 8 TESTS/VERIFICATION Provide checks: - Verify no analytics requests fired before consent (network test steps) - Verify cookie banner shows expiry info - Verify sitemap includes required URLs - Verify footer shows address on all marketing pages. OUTPUT FORMAT Return only: SUMMARY (≤7 bullets) CODE_CHANGES (file paths + code blocks) CONSENT_MODEL (stored fields + versioning) COOKIE_TABLE (what displayed) PAGES_ADDED (security/dpa outlines) SITEMAP (routes list + generation method) VERIFY_STEPS (manual checks) ROLLBACK_STEPS No extra commentary.

prompt 23: ROLE Senior product marketer + content systems engineer GOAL Build a production-grade content system for Lap: single positioning statement + one-liner enforced

everywhere, content inventory spreadsheet, duplicate-copy linting, CTA rules, copy style guide, glossary, claims registry with proof/hedging, persona matrix with unique value props, and page rewrites so each page has a unique JTBD and examples. AUDIT 2026-03-03-full-system-audit. RULES Reduce repetition to near zero. Every claim must be provable or hedged. Enforce with automation (lint/CI), not human memory. Keep outputs minimal and directly actionable. TASKS 1 POSITIONING + ONE-LINER Propose: - 1 Positioning Statement (exactly 1 sentence) - 1 One-liner (8–12 words) Fit Lap: “telemetry easy accessible giving immediate professional-grade results”. Provide final choices only (no variants) and a short “usage spec” mapping: hero line, meta description opening, pricing intro, nav tagline, OG card subtitle, email header preheader. 2 ENFORCEMENT MECHANISM Implement a single source of truth file for copy constants (e.g., content/brand.ts or content/brand.json) and update marketing templates to reference it for hero/meta/pricing/nav/OG/email headers. Ensure no duplicated literals remain. 3 CONTENT INVENTORY SPREADSHEET Create a spreadsheet template (CSV or XLSX) with columns: url, page_name, section_id, component_name, copy_block_id, owner, last_updated, persona_target, jtbd, primary_cta_label, primary_cta_url, secondary_ctas, proof_links, claims_used, glossary_terms_used, uniqueness_hash, notes. Auto-populate with a script that scans routes and extracts headings + copy blocks ids. 4 NO-DUPLICATE COPY LINT (BUILD-TIME) Implement a lint rule/script that: - parses marketing pages (MD/MDX/TSX) - extracts H2/H3 text and following paragraph blocks - normalizes whitespace/punctuation/case - flags duplicates above threshold (exact match OR similarity >X%) - prints file paths + offending blocks - fails CI on main. Provide configuration (ignore list for legal boilerplate). 5 CTA RULES Enforce one primary CTA per page: - add a linter that counts primary button components per route - fail build if >1 primary CTA - allow multiple secondary links. Update components to require explicit variant (primary/secondary) so counting is reliable. 6 COPY STYLE GUIDE Produce a compact style guide document (COPY_STYLE.md) covering: tone, banned words list, allowed claims policy, hedging policy (“may/help”), punctuation rules, casing rules, number formatting, units, and link style. 7 GLOSSARY Produce GLOSSARY.md with canonical definitions for key telemetry terms (delta, apex, braking point, turn-in, minimum speed, throttle pickup, trail braking, grip utilization, racing line, sector). Include “use this phrasing” + “avoid this phrasing”. 8 CLAIMS REGISTRY Create CLAIMS_REGISTRY.csv with fields: claim_id, claim_text, page_locations, proof_url(s), proof_type (internal test/user quote/benchmark), status (approved/needs_hedge/rejected), owner, last_reviewed. Update copy to reference claim_id tags where claims appear. CI should fail if a claim appears without claim_id or proof/hedge. 9 PERSONA MATRIX Create PERSONA_MATRIX.md mapping personas to unique value props and examples: rental kart, trackday, sim, teams/venues. For each: pains, desired outcome, product angle, proof type, recommended CTA, forbidden messages (to avoid generic overlap). 10 PAGE JTBD REWRITE PLAN Create a “rewrite plan” file listing each marketing page with: unique JTBD, unique example(s), unique proof element, and primary CTA. Ensure no two pages share the same JTBD sentence. OUTPUT FORMAT Return only: POSITIONING (positioning + one-liner + usage spec) FILES_TO_ADD (paths + short contents outlines) LINT_SPECS (duplicate-copy + CTA linter rules) SPREADSHEET_TEMPLATE (columns + sample row) CLAIMS_REGISTRY_TEMPLATE (columns + sample row) PERSONA_MATRIX (table) REWRITE_PLAN (page → jtbd → example → CTA) CI_WIRING (where to hook linters) ROLLBACK_STEPS No extra commentary.

prompt 24: ROLE Senior product marketer + content systems engineer GOAL Eliminate marketing copy repetition and enforce brand/copy consistency: unique benefit bullets per page, standardized CTAs/product+plan naming, deploy QA checklist, internal link map, legal date consistency, punctuation+units+numbers normalization, and a numbers policy. RULES Enforce via automation (lint/CI) where possible. Keep templates minimal. No per-page manual policing. TASKS 1 UNIQUE BENEFIT BULLETS Build a benefits registry: each “benefit bullet” has a unique id and allowed page(s). Implement a build-time linter that flags reused benefit bullet ids across pages unless explicitly allowed with a “framing override” tag. For pages that need similar benefits, require unique framing text and a different id. 2 CTA STANDARDIZATION Define a canonical CTA dictionary with exact casing: - Start free - See pricing - Request demo - Open app - Contact sales Enforce via lint: any button/link with CTA role must use one of these strings (or be in allowlist). Replace nonconforming CTAs. 3 PRODUCT NAMING STANDARDIZATION Define naming rules and enforce: - Product brand: choose one canonical name (Lap) and one legal/entity name (Lap Coach if required). - Feature tier label: World-Class reserved for the top tier only. Implement lint to detect forbidden variants and mixed terms in marketing content. Add a single source-of-truth NAME_TOKENS file and reference in templates. 4 PLAN NAMING STANDARDIZATION Define plan names for B2C only: Demo / Racer / World-Class. Define B2B names separately (e.g., Venue Pilot / Venue Pro / Enterprise) and forbid cross-mixing. Enforce via lint: B2C pages may not mention B2B plan tokens and vice versa unless on a dedicated comparison page. 5 CONTENT QA CHECKLIST Create a required pre-deploy checklist (CONTENT_QA.md) and enforce via PR template checkbox. Checklist must include: spelling/grammar pass, link check, uniqueness lint pass, CTA lint pass, claim proof/hedge check, naming lint pass, units/numbers policy pass, legal dates present. 6 INTERNAL LINK MAP Add a script that crawls internal links across marketing pages and outputs: - orphan pages (no inbound links) - dead links - loops (same 2–3 pages linking cyclically without path to CTA) - overlinked pages (too many outbound) Generate report artifact in CI and fail on dead links + orphans. 7 LEGAL DATES CONSISTENCY Add “Effective date: YYYY-MM-DD” and “Last updated: YYYY-MM-DD” to all legal pages. Enforce formatting with lint. Ensure dates are ISO and consistent across pages. 8 PUNCTUATION NORMALIZATION Define punctuation policy: - use em dash “—” not double hyphen “--” - choose quote style (“ ” or “ ” vs “ ”) and enforce - ellipsis “…” not “...” - decimal separator per locale (EN uses “.”, FR uses “,”) and enforce by page locale Implement lint/autofix script for common cases. 9 UNITS + NUMBERS NORMALIZATION Define units policy: - use km/h (not kph) - time: 1:23.45 or 83.45s (pick one format per context and document) - use ms only for sub-second engineering contexts; marketing uses seconds with 2 decimals - deltas precision: 0.01s; grip: 1% increments Implement a numbers formatter helper for marketing examples and enforce in content via lint (disallow raw “kph”, inconsistent decimals). OUTPUT FORMAT Return only: CTA_DICTIONARY (exact strings) NAMING_RULES (canonical terms + forbidden list) PLAN_RULES (B2C vs B2B tokens) BENEFITS_REGISTRY_SPEC (id format + allowed pages) LINTER_SPECS (benefits/cta/naming/legal/punctuation/units) QA_CHECKLIST (bullets) LINK_MAP_SCRIPT_SPEC (inputs/outputs/fail criteria) LEGAL_DATE_TEMPLATE (exact snippet) NUMBERS_POLICY (rules) CI_WIRING (where to run) ROLLBACK_STEPS No extra commentary.

prompt 25: ROLE Senior product content + i18n engineer GOAL Build a production-grade content system: microcopy library for UI strings, remove placeholder copy from prod, feature capitalization policy, translation readiness rules, persona tone toggles only if justified, strict marketing page templates, FAQ de-duplication, /insights freshness plan, content approval workflow, and automated spell/grammar checks in CI. AUDIT 2026-03-03-full-system-audit. RULES Enforce via automation (lint/CI) where possible. Keep outputs minimal and implementable. No duplicated FAQs or placeholder text in production. TASKS 1 MICROCOPY LIBRARY Create a single source-of-truth microcopy library file(s) (e.g., content/microcopy/en.json + fr.json stub) covering: - error titles/messages (auth, billing, upload, analysis) - empty states (no sessions, no laps, no comparison) - tooltips (delta, optimal lap, confidence) - banners (past_due, degraded analysis) Each entry: key, default string, optional vars, tone notes. Update UI components to reference keys instead of inline strings. 2 PLACEHOLDER COPY ELIMINATION Implement build-time lint that fails if any production route contains placeholder tokens: “lorem”, “ipsum”, “coming soon”, “TODO”, “TBD”, “placeholder”. Include allowlist for internal dev-only pages. Add a CI check and run it on main. 3 FEATURE CAPITALIZATION POLICY Define a canonical list of feature names + casing (e.g., Corner Coaching, Optimal Lap, Time Loss, Confidence Heatmap). Implement lint that flags mismatched casing in marketing + app copy and suggests canonical replacement. Store canonical list in content/feature_names.json. 4 TRANSLATION READINESS RULES Implement i18n readiness lint for TSX/MDX: - forbid string concatenation for user-facing text (e.g., "Hello " + name) unless using i18n interpolation - forbid embedded HTML inside translation strings; require component-based formatting - require all UI strings to be referenced via translation key function (t("…")) or microcopy lookup - ensure variables use named placeholders ({{name}}) - prohibit hardcoded locale-specific number/unit formatting; require formatter helpers. 5 TONE TOGGLES BY PERSONA Add a persona-tone policy: default single voice. Allow per-persona tone only if: - persona has distinct buying context AND - copy differs meaningfully (not synonyms). Implement by structuring content blocks per persona only on designated pages; otherwise reuse same base copy. Add a rule: no more than 2 tone variants per page. 6 STRICT PAGE TEMPLATES Create a marketing page template component enforcing section order: Hero → Proof → HowItWorks → Features → CTA → FAQ. Implement lint that: - checks required sections exist (by component or MDX markers) - checks order - fails if missing or out of order (allow legal pages exempt). Provide starter templates for product/pricing/venues pages. 7 FAQ DE-DUPLICATION Implement FAQ registry: each question has id and canonical text. Lint ensures: - identical or near-duplicate questions do not appear across product/pricing pages - questions must be referenced by id from registry, not written inline - allow explicit reuse only if tagged as “global faq” with single canonical location. 8 /INSIGHTS FRESHNESS PLAN Create a minimal plan: cadence, topic buckets (product, telemetry science, case studies, venue ROI), internal link rules (each post links to product/pricing + 2 related posts), and quarterly audit checklist. Add an “insights index” with tags and auto-generated related links. 9 CONTENT APPROVAL WORKFLOW Add PR template changes: require “Content QA” checkbox for any change touching marketing/app copy. Implement CI check that fails if content files changed but PR does not include the checkbox keyword in description (if CI has access) OR enforce via CODEOWNERS review requirement for content paths. 10 SPELLCHECK/GRAMMAR IN CI Add automated checks for MD/MDX: - spellcheck dictionary with allowed brand terms - grammar

check (lightweight) - fail on high-confidence errors only. Provide config files and CI steps. OUTPUT FORMAT Return only: MICROCOPY_SCHEMA (keys/vars format) FILES_TO_ADD (paths) LINTER_SPECS (placeholder/feature-case/i18n/template/faq) PAGE_TEMPLATE_SPEC (required sections) FAQ_REGISTRY_SPEC (id+rules) INSIGHTS_FRESHNESS_PLAN (bullets) WORKFLOW (PR template/CODEOWNERS/CI gate) CI_SPELLCHECK (tools+configs) ROLLBACK_STEPS No extra commentary.

prompt 27: ROLE Senior UX engineer + accessibility specialist GOAL Standardize navigation and page structure across marketing + app: canonical nav (≤5 items) with no per-page variants, breadcrumbs for subpages, consistent page intro and section rhythm, correct heading hierarchy (exactly one H1), global skip link + focus styles, consistent footer on /app, standardized back navigation, and secondary sticky section tabs for long pages. AUDIT 2026-03-03-full-system-audit. RULES Enforce via shared layout components + lint checks. Accessibility first. No duplicate nav implementations. Deterministic spacing tokens. TASKS 1 CANONICAL NAV Define a single primary nav structure (≤5 items). Implement as a single config object (nav.ts/json) used by all layouts (marketing + app). Remove per-page overrides. Add lint to fail if any page defines its own primary nav. 2 BREADCRUMBS Add Breadcrumb component for subpages: - driven by route map config - consistent styling and spacing tokens - aria-label, list semantics - shows on subpages only (not home/top-level). Ensure marketing and app share the same component. 3 PAGE INTRO PATTERN Create PageIntro component enforcing: - exactly one H1 - one short subline (p) - fixed spacing tokens - optional badges (e.g., “Beta”) but consistent. Replace ad-hoc intros. Add lint: every page must use PageIntro (except legal pages allowed). 4 SECTION RHYTHM Define typography + spacing tokens: - max line length (e.g., 70ch) for body - consistent H2/H3 margins - consistent section padding - consistent list spacing. Implement via global prose styles or Section component. Add lint to flag direct H2/H3 with custom spacing classes outside tokens. 5 HEADING HIERARCHY FIX Enforce heading rules: - exactly one H1 per page - no H1→H3 jumps - headings must be sequential (H2 under H1; H3 under H2). Implement a build-time checker for MDX/TSX that parses heading levels and fails CI on violations. 6 SKIP TO CONTENT + FOCUS STYLES Add global “Skip to content” link: - first focusable element - visible on focus - targets main content id. Add consistent focus-visible ring styles across all interactive elements (buttons, links, inputs). Add axe test in CI. 7 CONSISTENT FOOTER ACROSS /app Add Footer component reused in marketing + app shell: - legal links (privacy, terms, cookies) - security/status link - contact email - physical address if required. Ensure same typography/spacing. 8 BACK NAV PATTERN Define BackNav component: - consistent icon, label (“Back”), placement (top-left under breadcrumbs), and keyboard focus. Replace all ad-hoc “back” buttons/links. 9 SECONDARY NAV / STICKY TABS For long pages: implement SectionTabs component: - generated from H2 anchors or explicit config - sticky on scroll, highlights active section - keyboard accessible - does not shift layout (reserved height). Use on pricing/product/docs-like pages. 10 VERIFICATION Provide quick audit commands/steps: - list pages missing H1 or with multiple H1 - screenshot diff of nav/footer across routes - lighthouse/axe checks for skip link and breadcrumbs. OUTPUT FORMAT Return only: NAV_SPEC (≤5 items + routes) COMPONENTS (Breadcrumb/PageIntro/Section/Footer/BackNav/SectionTabs) LINT_SPECS (nav override + heading hierarchy + spacing tokens) CODE_CHANGES (file paths + code blocks)

A11Y_CHECKS (what to run) MANUAL_VERIFY_STEPS ROLLBACK_STEPS No extra commentary.

prompt 28: ROLE Senior UX/SEO engineer GOAL Improve site IA + SEO consistency: add search or sitemap page, improve 404 recovery links, eliminate “click here”, define URL policy, enforce canonical redirects (www vs non-www + trailing slash), add canonical tags, normalize pricing layout (B2C then B2B with anchors), add top-of-page UX for long pages, standardize external-link icon behavior, group legal pages consistently, and ensure consistent contact entry point. AUDIT 2026-03-03-full-system-audit. RULES Enforce via shared components + build-time lint where possible. No per-page drift. SEO must be deterministic. TASKS 1 SEARCH OR SITEMAP PAGE If page count >12, implement at least one of: - /sitemap page listing all key pages grouped (Product, Pricing, Company, Legal, Resources) with short descriptions and CTAs OR - simple client-side search over that page index (optional). Generate the page index from a single routes registry. 2 404 PAGE RECOVERY Update 404 page to include clear recovery links: Product, Pricing, Docs/Help, Login/App, Contact. Ensure consistent styling and a primary CTA. 3 NO “CLICK HERE” LINT Implement lint that fails build if internal links contain “click here”, “here”, “learn more” without context. Provide autofix guidance. 4 URL POLICY Define URL policy doc and enforce: - lowercase slugs - hyphens not underscores - no duplicates/aliases unless explicitly redirected - trailing slash policy (choose one) - localized routes rules if any. Implement redirect rules accordingly. 5 WWW VS NON-WWW CANONICAL Enforce single canonical host (choose www or apex). Implement: - Cloudflare/Page/Worker redirects to canonical host - canonical tags match - sitemap uses canonical URLs. 6 CANONICAL TAGS Ensure every public page sets canonical tag. Implement in shared SEO component/layout so it cannot be omitted. Exempt only truly non-canonical pages (e.g., previews). Add lint ensuring SEO component used. 7 PRICING LAYOUT Normalize Pricing page: - B2C section first (Demo/Racer/World-Class) - B2B section separated with clear heading and anchor link (#b2b) - sticky section tabs optional - keep CTAs consistent and avoid mixing plan names. 8 TOP-OF-PAGE UX Add consistent “Back to top” pattern on long pages: - appears after scroll threshold - anchors to top with accessible label - respects reduced motion. Use shared component. 9 EXTERNAL LINK ICON Standardize external-link behavior: - only external links show icon - external links open in new tab (target=_blank) and use rel=noopener - internal links never open new tab - icon placement consistent and aria-hidden. Enforce via Link component wrapper and lint if raw used. 10 LEGAL NAV GROUPING Create LegalNav group with consistent typography scale and order: Privacy, Terms, Cookies, Security, DPA. Use across legal pages and footer. 11 CONTACT ENTRY POINT Ensure header and footer both have “Contact” with same copy and URL. If using email, ensure consistent mailto text; if using page, link to /contact. OUTPUT FORMAT Return only: ROUTES_REGISTRY_SPEC (single source of truth) PAGES_TO_ADD (sitemap/search, 404 updates) URL_POLICY (rules) REDIRECTS (www/non-www + trailing slash) SEO_COMPONENT (canonical enforcement) LINT_SPECS (click here + seo usage + external link rules) PRICING_STRUCTURE (sections + anchors) UI_COMPONENTS (back-to-top + external link icon) MANUAL_VERIFY_STEPS ROLLBACK_STEPS No extra commentary.

prompt 29: ROLE Senior UX + growth engineer GOAL Add trust and conversion UX: a changelog page styled like marketing and linked from app, a help/docs link in app shell with

consistent placement, a consistent demo request funnel (hero CTA → form → confirmation), and standardized success pages with identical structure. AUDIT 2026-03-03-full-system-audit. RULES Single shared components. No per-page styling drift. Funnel must be measurable. Success pages must share one template. TASKS 1 CHANGELOG PAGE Create /changelog page using the marketing design system (typography/spacing/components). Content source: - simple MD/MDX entries OR - JSON feed, with version/date/tags/summary. Add “Changelog” link in app shell (footer or help menu) and in marketing footer. 2 HELP/DOCS LINK Add a “Help / Docs” link in app shell in a consistent placement (header right or user menu). Link to /docs if exists, otherwise to a minimal /help page with: getting started, FAQ, contact, and status link. Ensure same link appears in marketing footer. 3 DEMO REQUEST FUNNEL Implement a single funnel path: - Hero CTA “Request demo” always routes to /demo - /demo contains form (name, email, company/venue, use case persona, message) with validation and privacy note - On submit: create lead in backend/email (Resend/CRM) and redirect to /demo/success with state token (no PII in URL). Add tracking events: demo_cta_click, demo_form_view, demo_submit_success, demo_submit_fail. 4 SUCCESS PAGE TEMPLATE Create a reusable SuccessPage template component and use it for: demo success, signup success (if any), checkout success (if applicable), contact success. Template structure: - Title - 1–2 line summary - Next steps list (3 bullets max) - Primary CTA + Secondary link - Support/contact note. Ensure consistent styling and spacing across all success pages. 5 LINKS + NAV Ensure /demo is reachable from: - marketing hero CTA - pricing page - footer contact area. Ensure app shell also links to demo for B2B users if relevant (optional). 6 TESTS Add: - route exists and renders - form validation works - submission flow redirects to success - success template renders consistent sections - changelog renders entries and is linked from app shell. OUTPUT FORMAT Return only: PAGES (routes + purpose) COMPONENTS (Changelog, HelpLink placement, DemoForm, SuccessPage template) DATA_SOURCE (changelog format + lead submission endpoint) TRACKING_EVENTS (names) CODE_CHANGES (file paths + code blocks) TESTS_ADDED (list) MANUAL_VERIFY_STEPS ROLLBACK_STEPS No extra commentary.

prompt 30: ROLE Senior design systems engineer GOAL Finalize and enforce a production-grade typography system across marketing + app: single font stacks (display/body/mono), no mixed usage, tokenized type scale/line-heights/spacing, rem-only sizing, max line length, mono for numeric/code tokens, consistent casing/punctuation/emphasis/labels/links/quotes, WCAG AA muted contrast, consistent font loading, visual regression snapshots, locale-aware number/unit formatting, mobile wrapping rules, and contract tests preventing raw px. AUDIT 2026-03-03-full-system-audit. RULES Single source of truth (tokens). No per-page overrides. Enforce via lint/tests/CI. Minimal but complete. TASKS 1 FONT STACK DECISION Pick exactly 1 font each: Display, Body, Mono. Define fallback stacks. Ensure same across marketing + app. Add global CSS variables and apply via root theme. Remove any other font imports/usages. 2 TOKENS Create typography tokens: - font families - font sizes scale (in rem) - line heights - letter spacing (headings) - font weights - spacing tokens for intro blocks (H1+subline+CTA) - max line length token (ch) - link styles - label styles - tooltip max width. Implement in one file (e.g., tokens/typography.ts + CSS vars). 3 APPLY TOKENS Map every component type to tokens: headings, body, labels, buttons, tables, forms, tooltips, charts text. Ensure numeric fields and code-like tokens use mono font. Add utility

classes or component props to enforce. 4 REM-ONLY + BASE SIZE Set base font size once (html { font-size: 16px; }) and convert all sizes to rem. Add lint/contract test forbidding raw px font-size/line-height in components (allow borders). 5 CONSISTENCY RULES Enforce: - button casing (choose Sentence case OR Title Case) - headings punctuation (no trailing periods) - heading letter-spacing token usage - emphasis rules (when to use bold vs accent vs underline) - label typography rule (case + tracking + size) - hyperlink style and hover/focus consistent - quote/testimonial styling token. 6 CONTRAST FIX Adjust muted text token to meet WCAG AA against backgrounds used. Add automated contrast check (or snapshot + manual threshold). 7 FONT LOADING Ensure consistent loading: self-host or stable provider; preload key fonts; subset; font-display swap; avoid layout shift. Confirm no route uses different font config. 8 LOCALE FORMATTING Implement locale-aware formatter helpers for: - decimals - seconds formatting - percent spacing - unit spacing - FR punctuation spacing rules. Replace ad-hoc formatting in UI. 9 MOBILE WRAP + CTA NO-WRAP Add rules: - heading wrapping allowed but avoid manual ; set max widths and responsive sizes - CTA buttons use nowrap + min-width or shorter labels. 10 VISUAL REGRESSION Add typography snapshot tests for key pages (marketing home/pricing + app session page + billing). Use Playwright screenshots with stable fonts. 11 TYPOGRAPHY CONTRACT TEST Add CI test that fails if any component introduces: - font-size in px - non-token font-family - non-token line-height - multiple monospace fonts. OUTPUT FORMAT Return only: FONT_STACK (display/body/mono + fallbacks) TOKENS (names + values) COMPONENT_MAPPING (component → tokens) CSS_CHANGES (file paths + snippets) FORMATTERS (helpers + usage rules) LINT_TESTS (contract checks) VISUAL_REGRESSION (pages + commands) A11Y_CONTRAST (what changed) MANUAL_VERIFY_STEPS ROLLBACK_STEPS No extra commentary.

prompt 31: ROLE Senior design systems + frontend platform engineer GOAL Create and enforce a production-grade design system for Lap: one token source for colors/radii/shadows/borders/spacing/typography, token-only usage (no raw hex), elevation/border/radius systems, single icon set + sizes, button variants + active states, consistent focus/touch targets, standardized form errors/toasts/skeletons/empty states, responsive charts + mobile-first session views, reduced-motion, animation rules, OG card + favicon system, imagery framing, trust/partner logo styling, premium accent governance, chart palette tokens, consistent color semantics + optional color-blind palette, link contrast rules, spacing rhythm + grid system, and visual regression testing. AUDIT 2026-03-03-full-system-audit. RULES Single source of truth. Enforce via lint/tests/CI. No per-page overrides. Accessibility and mobile-first required. TASKS 1 SINGLE TOKEN FILE Create one token file (JSON or TS) defining: - colors (semantic + raw palette) - radii - shadows/elevation - borders (width + color) - spacing scale - typography tokens (font families/sizes/line-heights/letter-spacing) - z-index scale (optional) Export tokens to: CSS variables + Tailwind config (if used) + TS typings. 2 TOKEN ENFORCEMENT Add lint rules: - forbid hardcoded hex/rgb/hsl outside token file - forbid non-token radii/shadows/border widths - forbid inline style for design props unless whitelisted. CI fails on violations. 3 ELEVATION/SHADOW SYSTEM Define subtle elevation levels (e0–e4) with consistent usage rules (cards, modals, dropdowns). Replace ad-hoc shadows. 4 BORDER SYSTEM Define border widths (1px, 2px) and border colors (default, muted, focus, error). Replace random

border widths. 5 RADIUS MAP Lock radius scale with 8px as default (e.g., r0=0, r1=4, r2=8, r3=16). Replace random radii. 6 ICON SYSTEM Choose one icon set (Lucide recommended). Define allowed sizes (16/20/24) and mapping per component type. Add lint to forbid importing other icon packs or non-standard sizes. 7 BUTTON SYSTEM Define button variants: primary/secondary/ghost/destructive + sizes. Add pressed/active states + disabled states + loading states. Enforce via Button component only; lint forbids custom button styling outside. 8 FOCUS + TOUCH TARGETS Define global focus-visible ring tokens and apply to all interactive components. Enforce minimum 44px touch targets (padding/min-height) via component primitives. Add a test/lint for interactive elements with too-small hitboxes (best-effort). 9 FORMS Standardize form error styling: - error color token + border token + helper text placement - ARIA role=alert for error messages - consistent spacing and typography tokens. 10 TOASTS Standardize toast system: - stacking position - duration defaults by severity - dismissible behavior - max one action button - accessible announcements. 11 LOADING + EMPTY STATES Standardize skeleton components (no spinners except tiny inline) and empty state pattern: icon + title + hint + CTA. Provide primitives and require usage for lists/dashboards. 12 RESPONSIVE CHARTS + MOBILE SESSION VIEW Ensure charts resize with ResizeObserver and never overflow. Define chart container tokens (min-height/aspect) to prevent CLS. Make session pages mobile-first (trackside): key KPIs top, tabs/accordions for secondary panels. 13 REDUCED MOTION + ANIMATION RULES Implement global reduced-motion honoring prefers-reduced-motion: disable Lenis/GSAP/Three heavy animations and use minimal transitions. Define animation tokens: durations + easings + allowed contexts; forbid motion on reading pages (pricing/terms). 14 BRAND ASSETS SYSTEM Define OG card design system: per-page OG templates with tokens. Define favicon/touch icons (monochrome variants) aligned to final mark. 15 IMAGERY SYSTEM Standardize imagery framing: screenshot frames, shadows, background, radius. Standardize trust badges and partner logo grid (size/opacity/spacing). Define premium accent token (gold) with usage limits and lint (only allowed in defined components). 16 COLOR SEMANTICS + OPTIONAL COLOR-BLIND PALETTE Define semantic colors (success/warn/error/info/brand) and enforce meaning rules (red only destructive/errors). Define chart palette tokens (series1..n, alert). Optionally add alternate color-blind palette toggle (pref stored) and ensure contrast. Ensure link styles meet contrast and not color-only (underline on hover/focus). 17 GRID + SPACING RHYTHM Define breakpoints, columns, gutters, section padding rhythm. Replace ad-hoc spacing across pages with Section/Layout primitives. 18 VISUAL REGRESSION Add visual regression for key marketing pages + key app screens using Percy or Chromatic (or Playwright screenshot diffs). Gate merges on significant diffs. OUTPUT FORMAT Return only: TOKENS_FILE_SPEC (schema + example keys) EXPORTS (CSS vars + tailwind/ts) LINT_RULES (what forbidden + configs) COMPONENT_PRIMITIVES (Button/Icon/Form/Toast/Skeleton/EmptyState/Layout) MOTION_RULES (reduced-motion + animation tokens) BRAND_ASSETS (OG/favicon rules) IMAGERY_RULES (frames/badges/logos) COLOR_SEMANTICS (meaning + chart palette) GRID_SYSTEM (breakpoints/gutters) VISUAL_REGRESSION (tool + pages + commands) MANUAL_VERIFY_STEPS ROLLBACK_STEPS No extra commentary.

prompt 32: ROLE Senior accessibility engineer GOAL Raise Lap to production-grade accessibility and semantic HTML correctness: alt text rules, labeled forms, announced errors,

keyboard navigation, modal focus trap, stable tab order, semantic headings/lists/tables, icon ARIA rules, skip link, reduced-motion disables smooth scroll inertia, WCAG AA contrast, accessible chart summaries, correct lang + hreflang, accessible cookie banner, noopener on target=_blank, export UI context, and automated axe audits in CI. AUDIT 2026-03-03-full-system-audit. RULES Fix via shared primitives + lint/tests. No one-off patches. CI must block regressions. TASKS 1 ALT TEXT POLICY Implement lint rules for images: - decorative images must have alt="" and aria-hidden if needed - meaningful images must have descriptive alt (non-empty) - forbid missing alt. Add helper component enforcing these rules. 2 FORM LABELS Enforce explicit labels for all inputs: - label element with htmlFor or aria-label only when no visible label is appropriate - forbid placeholder-only labeling. Add FormField primitive that renders label + helper + error consistently. Add lint to flag inputs without labels. 3 ANNOUNCED ERRORS Ensure error messages use role="alert" (or aria-live=assertive) and are programmatically associated with fields via aria-describedby. Ensure global toasts also announce. 4 KEYBOARD NAV Ensure menus/tabs/dropdowns are keyboard operable: - ESC closes - outside click closes - focus returns to trigger - arrow key navigation where appropriate. Use a single menu/dialog library (Radix recommended) and remove custom ad-hoc implementations. 5 MODAL FOCUS TRAP Ensure all modals trap focus and restore focus on close. Prevent background scroll. Validate nested modals behavior. 6 TAB ORDER CONSISTENCY Remove tabindex hacks. Ensure DOM order matches visual order. For long pages, ensure skip link + landmarks (, ,) are present. Add tests for focus order on key screens. 7 SEMANTIC STRUCTURE Enforce: - headings used semantically (no div-as-heading) - no heading level jumps and exactly one H1 per page - lists use / - tables use and captions when helpful. Add MDX/TSX semantic lint rules. 8 ICON ARIA POLICY Standardize icon component: - decorative icons aria-hidden="true" focusable="false" - functional icons have accessible name via aria-label on button/link or sr-only text. Lint raw svg usage. 9 SKIP LINK Ensure skip link visible on focus, targets main content id, works on marketing + app. Add regression test. 10 REDUCED MOTION Ensure prefers-reduced-motion disables: - smooth scroll inertia (Lenis) - heavy animations (GSAP/Three) - parallax effects. Keep minimal transitions. 11 CONTRAST Ensure all text meets WCAG AA including muted tokens. Add token-level contrast checks and/or snapshot audit. Fix tokens not components. 12 CHART ACCESSIBILITY Add accessible summaries for charts: - provide textual key stats (min/max/avg, delta) - aria-describedby linking chart to summary - ensure keyboard focusable legend if interactive. 13 LANG + HREFLANG Ensure set per locale page. When multilingual ships, add hreflang tags and x-default. Add lint to require lang and hreflang completeness for localized routes. 14 COOKIE BANNER ACCESSIBILITY Ensure cookie banner is keyboard accessible: - focus managed on open - can tab through, ESC closes if modal - buttons have clear labels - preferences panel accessible. 15 TARGET=_BLANK SAFETY Enforce rel=noopener (and noreferrer if desired) for all target=_blank links via Link component wrapper + lint for raw . 16 EXPORT UI CONTEXT Ensure download/export UI includes: - file name - content context (session name/date) - chart titles included in exported images/PDFs - accessible labels for download buttons. 17 CI ACCESSIBILITY AUDITS Add axe audits in CI for marketing + key app routes (logged in state): - run Playwright + axe on routes: home, pricing, login, sessions list, session detail, billing - fail on serious/critical issues. Produce artifact report. OUTPUT FORMAT Return only: POLICIES (alt/forms/icons/links) COMPONENT_PRIMITIVES

(AppImage/FormField/Dialog/Menu/Link/ChartSummary) LINT_SPECS (rules + fail conditions) CODE_CHANGES (file paths + snippets) CI_AXE (routes + commands) MANUAL_VERIFY_STEPS (keyboard checklist) ROLLBACK_STEPS No extra commentary.

prompt 33:
ROLE Senior SEO + content systems engineer GOAL Make Lap's marketing SEO production-grade: enforce meta title/description patterns, unique per-page OG fields + images, add structured data, maintain complete sitemap, canonical URL consistency, noindex for /app, internal linking rules to prevent keyword cannibalization, build-time content uniqueness gate, and a quarterly content audit routine. AUDIT 2026-03-03-full-system-audit. RULES Single source of truth for SEO metadata. Enforced by CI. No per-page drift. No duplicated OG/meta copy. TASKS 1 META TITLE SYSTEM Implement a meta title generator: pattern "Lap | {PageName} | {PrimaryKeyword}". Store per-page PageName + PrimaryKeyword in a routes registry. Enforce max length (e.g., 60 chars). Add lint: every public page must use SEO component and must provide PageName+PrimaryKeyword (or registry entry). 2 META DESCRIPTION SYSTEM Implement meta description generator: unique per page, <160 chars, includes CTA (from a canonical CTA list). Store per-page description in registry or content frontmatter. Lint: must be unique (no reuse), length limit, must contain CTA verb, no forbidden placeholders. 3 OG FIELDS + OG IMAGE SYSTEM For each marketing page require unique OG:title and OG:description (not reused). Generate per-page OG:image using a templated system: one base design + per-page title/subtitle. Store OG config in registry. Lint: no duplicates across pages. Ensure images are generated at build-time and referenced correctly. 4 STRUCTURED DATA Add JSON-LD where relevant: - Organization (site-wide) - Product (product page) - Offer/Pricing (pricing page) - FAQPage (where FAQs exist) Implement via SEO component with page-type flags. Validate with schema checks. 5 SITEMAP COMPLETENESS Generate sitemap.xml from routes registry; include all marketing pages; exclude /app and any noindex pages. Ensure it updates automatically on route changes. Add CI check: sitemap contains every route marked "public". 6 CANONICAL URL CONSISTENCY Choose canonical host (www or apex) and trailing slash policy. Implement: - redirect to canonical at edge (Cloudflare/Next) - canonical tag matches - sitemap uses canonical. Add tests verifying canonical tags and redirects. 7 /APP NOINDEX Ensure /app routes include meta robots noindex,nofollow and are excluded from sitemap. Keep robots.txt aligned. Add CI check: any route under /app has noindex meta. 8 INTERNAL LINKING + KEYWORD CANNIBALIZATION RULES Define linking rules per "primary keyword" page: - product page targets keyword A, pricing targets keyword B, insights target long-tail C - insights must link to product/pricing with varied anchor text; avoid overusing the same anchor repeatedly - prohibit multiple pages claiming the same primary keyword unless explicitly intended. Implement lint: no two pages share same PrimaryKeyword unless allowlisted; check anchors for overuse. 9 CONTENT UNIQUENESS TEST Implement build-time uniqueness gate: - split each page into normalized blocks (headings + paragraphs + bullets) - compute similarity between pages - fail build if any pair shares >X% identical blocks (configurable) excluding allowlisted boilerplate (footer/legal snippets). Output offending blocks and file paths. 10 QUARTERLY CONTENT AUDIT ROUTINE Create CONTENT_AUDIT.md + a runnable script that checks: - broken internal/external links - missing/duplicated meta/OG - outdated screenshots (age threshold or referenced asset version)

- repeated claims without proof registry entry - drift from naming/CTA policy - sitemap/canonical/noindex integrity. Add a scheduled CI workflow (quarterly) that opens an issue with the report. OUTPUT FORMAT Return only: ROUTES_REGISTRY_FIELDS (page_name, primary_keyword, meta_desc, og fields, canonical) SEO_COMPONENT_SPEC (how applied) OG_IMAGE_PIPELINE (how generated) JSON_LD_MAP (page → schema types) SITEMAP_RULES (include/exclude) CANONICAL_POLICY (host + slashes + redirects) LINT_SPECS (meta/og/keywords/noindex/uniqueness) AUDIT_ROUTINE (checks + schedule) CI_WIRING (jobs) ROLLBACK_STEPS No extra commentary

Prompt 34 — Event-based corner timeline (replace 20/60/20 splits)
You are a senior motorsport telemetry engineer. In this repo, replace the current fixed corner phase splitting with an event-based “Corner Timeline” per corner on the s-axis. Implement detection of: Braking Point (BP), End of Braking (EOB), Max Rotation Point (MRP=peak yaw_rate), Min-Speed point, Geometric Apex proxy (inside-most lateral position vs ref line if available), Throttle-On (or accel-positive), Track-Out. Requirements:

- Work with missing channels: if brake pressure absent, infer BP/EOB from longitudinal accel; if yaw_rate absent, approximate from heading derivative; if position absent, skip apex/track-out and mark unavailable.
- Add dataclass CornerTimeline and persist to corner analysis outputs (API + report).
- Update corner_analysis to compute losses per event-to-event segment (BP→EOB, EOB→MRP, MRP→ThrottleOn, ThrottleOn→TrackOut) with confidence scores.
- Integrate into pipeline stage 6/10 and web endpoints /corners and /corner-replay without breaking existing schema (add fields, don’t remove).
- Add unit tests using synthetic laps (known event placements), plus regression tests for existing corner counts.
  Files to inspect/modify: telemetry_tool/corner_detection.py, corner_analysis.py, derived_channels.py, pipeline.py stage 6/10, web/routes/data_v2.py, reporting_html.py.

Prompt 35 — Robust MRP detection + MRP-based mistake classifiers
Implement a robust MRP detector and add MRP-driven mistake detection. Definition: MRP is the dominant yaw-rate peak within a corner window; should be near min speed and around end-of-braking for most corners. Requirements:

- Compute MRP(s), MRP strength (peak prominence), and MRP stability (lap-to-lap std-dev of s position).
- Add mistake classifiers (per corner, per lap and aggregated): “no clear MRP / plateau”, “MRP drift”, “MRP after apex proxy”, “power before rotation” (throttle/accel increases while yaw-rate still below threshold), “rotation too late” (MRP too far after EOB).
- Output: per-corner flags + severity + evidence metrics (delta time cost, s offsets, yaw-rate curves summary).
- Add confidence gating based on channel availability and data_quality score; ensure graceful degradation.
- Expose results in /corners and reporting HTML; add to rank_improvement_opportunities weighting.

Files: corner_detection.py (event extraction), corner_analysis.py, deltas.py (time cost attribution), data_quality.py, pipeline.py stage 10c/10d, reporting_html.py. Add tests for edge cases: noisy yaw, multiple peaks, missing yaw.

Prompt 36 — Brake release quality metrics (with/without brake pressure)
Add a “Brake Release Quality” module and integrate into corner coaching. Requirements:

- If brake pressure channel exists: detect peak brake, release slope, “over-drop” (pressure drops too much too early), “late release/lock risk proxy” (high brake retained into low-speed region), and terminal trail pressure at Throttle-On (T%).
- If brake pressure missing: approximate using longitudinal accel and speed derivative; clearly label as inferred and lower confidence.
- Provide per-corner metrics: peak decel/pressure, time in brake zone, release linearity score, release timing relative to MRP, and a recommended adjustment (+/– meters earlier/later release; smoother/longer trail).
- Integrate into corner_analysis outputs and coaching hints; add filters so low-quality GPS doesn’t produce fake “release issues”.
- Add tests: synthetic brake traces + accel-only fallback; validate invariants (peak before EOB, monotonic release trends).
  Files: derived_channels.py, corner_analysis.py, features.py, pipeline.py, channel_registry.py (optional brake channel registration), reporting_html.py.

Prompt 37 — Brake–rotation coupling (“string theory”) scoring and corrections
Implement coupling scores that explain rotation management (understeer vs oversteer patterns) via brake-release vs yaw response. Requirements:

- Compute coupling score: correlation/lag between brake release and yaw-rate growth (or heading change rate) inside EOB window.
- Detect patterns:
  - Under-rotation: steering/curvature demand high but yaw response weak while brake released early.
  - Over-rotation: yaw spikes while brake still high; recommend earlier release / reduced trail.
- Add “Terminal Trail Braking Pressure (T%)” as a first-class feature (at Throttle-On or first accel-positive).
- Output coaching as specific action: “hold X% trail until yaw reaches Y”, “delay throttle-on until yaw crosses threshold”, “reduce brake earlier by Z meters”.
- Must work with missing signals: if yaw unavailable, skip coupling; if brake unavailable, skip coupling and mark unavailable.
- Add confidence and avoid unstable advice if corner window too short or noisy.
  Files: corner_analysis.py, derived_channels.py, data_quality.py, pipeline.py stage 10, reporting_html.py. Add tests: controlled signals with known lag/correlation.

Prompt 38 — Compound corner detection + “entanglement” analysis
Add a compound-corner system that links consecutive corners when there is no meaningful reset (short straights / continuous curvature). Requirements:

- Detect corner sequences (N corners) as compounds using s-gap thresholds and curvature continuity.
- Compute propagation metrics: how Corner i exit (MRP location, track-out, exit speed) impacts Corner i+1 entry (entry speed, BP shift, line deviation).
- Attribute time loss across the compound: move blame upstream when evidence shows Corner i caused Corner i+1 loss.
- Output: “compound_id”, per-corner “upstream dependency score”, and coaching that targets the earliest leverage point.
- Update rank_improvement_opportunities to surface compound-driven fixes (one recommendation can span multiple corners).
  Files: corner_detection.py (compound grouping), corner_analysis.py, deltas.py, reference_line.py/line_optimization if needed, pipeline.py, web endpoints and report. Add tests using synthetic multi-corner sequences.

Prompt 39 — “Trap / setup corner” detector (kinks before braking zones)
Implement detection of “setup corners” (kinks or mild bends) that primarily exist to position the car for a heavy braking corner. Requirements:

- Identify kinks preceding heavy braking zones by curvature + subsequent decel magnitude and short s-gap.
- Score whether the driver sacrificed the kink correctly to maximize the next braking zone (compare line deviation + entry positioning proxy + braking efficiency).
- Produce coaching: “do not apex kink”, “prioritize outside placement for braking”, with a measurable target (s-position of kink MRP or line deviation threshold).
- Integrate into corner replay: annotate kink and next braking zone as linked; expose in /corner-replay payload.
  Files: corner_detection.py, corner_analysis.py, deltas.py, reference_line.py/line_optimization.py, web/routes/data_v2.py, reporting_html.py. Add tests for false positives (normal corners not tagged as kinks).

Prompt 40 — Coaching output engine (trustworthy, structured, capped objectives)
Refactor coaching text generation to be consistent, evidence-based, and limited. Requirements:

- Build a coaching generator that outputs:
  - “What you did well” (confirming feedback) backed by metrics
  - “One key correction” per corner (only if confidence high)
  - “Top 3 objectives for next session” global cap=3, each with event target + change direction + measurable success metric
- Enforce a confirming:corrective ratio target (configurable) so reports don’t feel like endless criticism.

- Every advice item must include “evidence” fields: delta time estimate, s-offset, metric values, confidence.
- Integrate into HTML report and JSON API (add “coaching_v2” object; keep legacy fields).
  Files: corner_analysis.py (generate_coaching_hint), reporting_html.py, pipeline.py summary assembly, web/models.py persistence, web/routes/*. Add snapshot tests for deterministic outputs given fixed inputs.

Prompt 41 — Drill library + deliberate practice session builder
Create a drills layer that turns detected issues into a practice plan. Requirements:

- Build a drill registry (id, title, goal, setup, execution, telemetry success criteria).
- Map issues → drills: brake release over-drop, MRP drift, power-before-rotation, compound setup error, kink trap, etc.
- Generate a session plan: warm-up baseline, 1–2 drills, short evaluation runs, reflection prompt; output as JSON + HTML section.
- Use user constraints: if channels missing, pick drills that can still be measured; otherwise mark “needs brake sensor” etc.
- Expose via API endpoint /practice-plan (or embed in session summary), and add to report.
  Files: new telemetry_tool/drills.py, integrate in pipeline.py stage 10 summary, reporting_html.py, web/routes/sessions_v2.py. Add unit tests for mapping correctness and deterministic plan generation.

Prompt 42 — Condition clustering before optimal lap (stop blending incompatible laps)
Improve theoretical/optimal engine by clustering laps by conditions so envelopes aren’t blended fiction. Requirements:

- Derive condition features per lap: lap time, data_quality, grip proxy (max lat accel vs curvature), temperature if present, session meta tags, traffic proxy (micro-slowdowns / speed variance), wet proxy (reduced accel envelope).
- Cluster laps into a small number of “conditions” (rule-based first; optional k-means later) and pick the best cluster for theoretical generation.
- Persist chosen cluster id, cluster stats, and expose in theoretical_metrics + report.
- Ensure fallbacks still work (if too few laps, revert to current behavior but clearly label).
  Files: pipeline.py stage 10 selection logic, theoretical_engine.py (phase-aware/envelope binning), lap_simulation.py inputs, data_quality.py, web/models.py persistence. Add tests: synthetic sessions with two distinct grip regimes.

Prompt 43 — Build a mu(s) grip map from real laps and feed it into the solver
Add a distance-binned grip estimator mu(s) and integrate into theoretical speed caps. Requirements:

- Estimate per s-bin lateral capability from observed laps: use v^2 * curvature (or lat accel if available) and derive effective mu(s) with robust percentiles + outlier rejection.

- Produce mu_mean(s), mu_pXX(s), confidence per bin (sample count + variance + data_quality).
- Use mu(s) to cap cornering speed in lap_simulation.py and/or the adapter path (kart_envelope input if supported).
- Visualize mu(s) in report (simple plot) and expose via API.
- Must degrade gracefully: if curvature unreliable or bins sparse, revert to global mu with warning.
  Files: reference_line.py (curvature), theoretical_engine.py or new grip_map.py, lap_simulation.py, theoretical_adapter.py, reporting_html.py, web/routes/data_v2.py. Add unit tests: known mu(s) synthetic track.

Prompt 44 — Add a “line family” optimizer (early/late apex variants) + sequence optimization for compounds
Implement a first raceline optimizer without full boundaries: generate line variants per corner and pick min-time outcome. Requirements:

- For each corner (or compound), generate a small discrete set of line families: early/neutral/late apex via curvature shaping or offsetting the reference line within estimated track width.
- Recompute curvature profile and run lap simulation per variant; pick best feasible time.
- Include sequence optimization for compounds (optimize corner set jointly, not independently).
- Output: chosen line family per corner, expected delta vs baseline, and constraints hit rates.
- Keep runtime manageable: cache curvature arrays, limit variants, allow a config flag.
  Files: reference_line.py (line generation), lap_simulation.py (run many sims), pipeline.py stage 10, line_optimization.py (reuse if relevant), web persistence + map endpoints. Add tests: toy track with known best late-apex for exit-priority corner.

Prompt 45 — Estimate track boundaries from telemetry scatter and constrain the line optimizer
Add boundary estimation so raceline choices are physically plausible. Requirements:

- Using all lap XY traces, estimate left/right boundaries per s-bin (e.g., percentile envelopes perpendicular to reference line).
- Smooth boundaries and enforce minimum/maximum track width sanity constraints.
- Store boundary polylines in DB and expose via API; add map overlay for debugging.
- Update line-family generator to respect boundaries (offset limits) and avoid “outside track” solutions.
- Provide confidence per segment (sample count, variance).
  Files: reference_line.py (new boundary estimator), web/models.py (new tables or JSON blobs), web/routes/data_v2.py, reporting_html.py. Add tests: synthetic scatter around a centerline with known width.

Prompt 46 — Output an “optimal corner plan” (targets, not just a speed trace)
Extend theoretical outputs to include actionable per-corner targets: BP, EOB, MRP, Throttle-On, Track-Out, plus priority rationale (entry vs exit). Requirements:

- Based on the chosen theoretical lap + event detector, compute target event s-positions and target speeds at each event.
- Output per corner: target timeline, “why” label (exit-priority due to straight/next corner), and recommended driver change from current best lap (meters earlier/later, km/h target).
- Integrate into /corners and /corner-replay so UI can show “ghost targets”.
- Add validation: targets must be consistent with feasibility constraints (no throttle-on before required rotation threshold, etc.), otherwise downgrade confidence and warn.
  Files: corner_analysis.py, theoretical_engine.py/lap_simulation.py outputs, pipeline.py stage 10, web/routes, reporting_html.py. Add tests: deterministic targets on synthetic track.

Prompt 47 — Endurance/team mode analytics (stint, traffic, risk/reward)
Add an endurance-focused analytics layer for sessions with multiple stints/drivers. Requirements:

- Stint detection: segment session by pit/out-lap markers or large stop events; compute stint pace, consistency (lap std-dev), degradation trend.
- Traffic loss proxy: detect micro-lifts / delta spikes and estimate time lost to traffic per lap and per corner.
- Risk vs reward pass classifier (heuristic): identify corners where attempted gains increase variance/incident risk (use instability/yaw spikes/line deviation + small net delta).
- Team coaching: identify the weakest stint/driver and generate 3 targeted objectives using the same coaching_v2 format.
- Expose via API and report section “Endurance”.
  Files: pipeline.py (session-level), deltas.py, data_quality.py, corner_analysis.py, web/routes/sessions_v2.py, reporting_html.py. Add tests with synthetic stints and injected traffic events.

PROMPT 48 — Distance Integrity Preflight + Auto-Repair (s-axis hardening)
You are a senior telemetry engineer. Implement a mandatory “distance_m integrity” preflight before any resampling/corners/deltas/theoretical. Requirements:

- Compute metrics per lap: monotonic_ratio, jump_count, compression_ratio, lap_length, lap_length_zscore.

- If XY available: rebuild distance_m = cumulative sqrt(dx^2+dy^2), then normalize to lap start.

- If XY not available: apply monotonic regression to distance_m vs time_s and smooth small glitches; reject laps with large unrecoverable jumps.

- Persist a DistanceQuality object in session summary + API (add fields only).

- Gate advanced outputs (theoretical + coaching_v2 + sensitivity maps) if distance quality below threshold; downgrade confidence with reason codes.
  Files: telemetry_tool/resampling.py, pipeline.py (pre-stage 7), reference_line.py (XY helpers), data_quality.py, web/routes/data_v2.py, reporting_html.py.
  Tests:

- Synthetic lap with non-monotonic distance + XY repair path.

- Lap with big jump and no XY must be rejected/gated.
  Acceptance:

- No downstream module sees non-monotonic distance_m; gating is visible in API/report.

PROMPT 49 — Unified Optimal Lap Engine (single solver contract)
Implement a single "optimal lap" generation pipeline that always produces v_opt(s) via feasibility solver. Requirements:

- Define a unified input contract: curvature(s), v_cap(s), ax_bounds(v), ay_bounds(v), optional mu(s), optional power model.

- Make kart_envelope / lap_simulation / percentile envelope produce ONLY caps/params; never directly output final optimal speed trace.

- Always run a forward/backward feasibility solver (shared module) to compute v_opt(s).

- Store method provenance: cap_source(s) + solver_version.
  Files: telemetry_tool/theoretical_adapter.py, lap_simulation.py, theoretical_engine.py, pipeline.py stage 10, web/models.py persistence.
  Tests:

- Same session yields same v_opt(s) regardless of which cap path is used (given same caps).
  Acceptance:

- One canonical output schema; no conflicting optimal traces.

PROMPT 50 — Theoretical Confidence Score + Reason Codes (everywhere)
Build a single confidence score system for theoretical + coaching outputs. Requirements:

- Compute confidence_score 0–100 and confidence_reasons[] based on:
    - distance quality metrics, data_quality score, lap_count, cluster purity, channel coverage, solver convergence, curvature reliability.
- Persist in theoretical_metrics + coaching_v2 + corner outputs.
- UI/report must display score + top reasons.
  Files: data_quality.py, pipeline.py stage 10 summary assembly, web/models.py, reporting_html.py, web/routes/data_v2.py.
  Tests:
- Low lap count, missing yaw_rate, bad distance → confidence drops with correct reasons.
  Acceptance:
- Any advanced output includes confidence fields; no silent “low trust” results.

PROMPT 51 — Deterministic Pipeline Guarantee (reproducibility)
Make analysis outputs deterministic for identical inputs. Requirements:

- Fix seeds for any randomness (clustering, sampling).
- Enforce stable sorting of laps and stable distance grids.
- Ensure any “median overlap” or binning uses deterministic tie-breaks.
- Add snapshot regression tests for key outputs (theoretical, corners, deltas).
  Files: pipeline.py, resampling.py, theoretical_engine.py, corner_detection.py, any clustering module, tests/.
  Acceptance:
- Running analysis twice yields identical JSON payloads (byte-stable after sorting keys).

PROMPT 52 — Grade / Slope Estimation and Accel Correction
Add grade(s) estimation and correct longitudinal acceleration for slope bias. Requirements:

- If altitude present and stable enough: estimate grade(s) via smoothed dz/ds.
- Compute a_long_corrected = a_long_raw - g*sin(grade).
- Use corrected accel for braking/throttle event inference and accel envelope learning.

- Degrade gracefully if altitude is noisy/unavailable; add confidence reason.
  Files: derived_channels.py, data_quality.py, corner_analysis.py, theoretical_engine.py.
  Tests:

- Synthetic uphill/downhill segment where raw accel is biased; corrected accel restores expected values.
  Acceptance:

- Brake/throttle inference becomes less slope-sensitive; outputs label “grade used” or “grade unavailable”.

PROMPT 53 — Combined Friction Envelope (friction circle proxy)
Extend grip modeling beyond lateral-only mu(s). Requirements:

- Estimate per speed-bin envelope of sqrt(ax^2 + ay^2) (or separate ax/ay bounds) using robust percentiles.

- Provide friction_circle_limit(v) and/or ellipse parameters; quantify confidence by sample count and variance.

- Feed into feasibility solver to prevent impossible accel+cornering combos.
  Files: theoretical_engine.py (or new friction_envelope.py), lap_simulation.py, derived_channels.py.
  Tests:

- Synthetic dataset with known friction circle; recovered envelope matches within tolerance.
  Acceptance:

- Solver never produces states outside learned envelope; reason codes appear when envelope sparse.

PROMPT 54 — Proper GPS+IMU Fusion (EKF-lite)
Implement an EKF-lite state estimator to stabilize heading/yaw_rate/velocity. Requirements:

- Fuse GPS position/velocity with IMU accel/gyro (if present) into state: x,y,v,heading,yaw_rate (minimum).

- Output filtered channels: heading_f, yaw_rate_f, accel_long_f, accel_lat_f with covariance-based confidence.

- If IMU missing, fall back to smoothing-only but label as such.
  Files: new telemetry_tool/fusion_ekf.py, pipeline.py stage before derived channels,

derived_channels.py consumers, data_quality.py.
Tests:

- Synthetic noisy GPS + IMU: EKF reduces yaw_rate noise and improves curvature stability.
  Acceptance:

- Corner detection and MRP events become more stable with EKF available.

PROMPT 55 — Solver Diagnostics: Limiting Factor Map + Saturation
Add explainability to the feasibility solver. Requirements:

- For each s-bin, label limiting factor: cornering / braking / traction / power / cap.

- Compute saturation metrics per corner (e.g., % bins at >98% of active constraint).

- Persist in theoretical outputs; expose in report (simple stacked summary + per-corner breakdown).
  Files: lap_simulation.py (or shared solver module), theoretical_adapter.py mapping, reporting_html.py, web/routes/data_v2.py.
  Tests:

- Toy track where cornering clearly dominates; map reflects that.
  Acceptance:

- Users can see “why” optimal is limited at each corner.

PROMPT 56 — Uncertainty Bands for Optimal Trace (v_opt ±)
Add uncertainty quantification. Requirements:

- Build v_opt_low(s), v_opt_high(s) using:
  - mu_pXX(s) variance, envelope quantiles, distance/curvature reliability.

- Compute lap_time interval [t_low, t_high].

- Degrade gracefully (bands widen) when confidence drops.
  Files: theoretical_engine.py / grip_map.py, solver module, web/models.py, reporting_html.py.
  Tests:

- Sparse bins produce wider bands; high-quality sessions produce narrow bands.
  Acceptance:

- Report/API show bands; no overconfident single-number claims.

PROMPT 57 — Cross-Validation / Overfit Detector (session-level)
Implement internal validation to detect “too good to be true” theoretical. Requirements:

- Split laps into train/holdout (deterministic split).

- Learn caps/envelopes on train; compute plausibility scores on holdout:

    - how often holdout approaches theoretical in segments

    - whether theoretical exceeds realistic bounds everywhere

- Add confidence reason “overfit_suspected” if failing.
  Files: pipeline.py stage 10, theoretical_engine.py, data_quality.py.
  Tests:

- Contrived dataset where one outlier lap forces unrealistic envelope; CV flags it.
  Acceptance:

- Confidence decreases when theoretical is unsupported by real laps.

PROMPT 58 — Driver Style Fingerprint (session/persona features)
Create a driver style profile from laps. Requirements:

- Compute per session (and optionally per driver_id): brake aggressiveness, trail length, MRP timing, exit priority, line variance.

- Persist driver_style vector + summary labels (e.g., “late braker”, “exit-focused”) with confidence.

- Use it to personalize coaching phrasing and ranking weights (without changing core facts).
  Files: features.py, corner_analysis.py, pipeline.py summary, web/models.py.
  Tests:

- Synthetic profiles produce correct labels.
  Acceptance:

- Coaching becomes consistent per style; never contradicts evidence.

PROMPT 59 — Stability / Risk Index (global + per corner)
Implement a risk score that penalizes unstable “gains”. Requirements:

- Compute risk signals: yaw spikes, MRP plateau, line deviation spikes, accel oscillation, brake release instability.

- Output risk_index per corner and per lap; integrate into rank_improvement_opportunities (reward high gain / low risk).
  Files: corner_analysis.py, deltas.py, line_optimization.py, data_quality.py, reporting_html.py.
  Tests:

- Inject yaw noise and line deviation → risk increases.
  Acceptance:

- Recommendations prefer stable gains; risky corners are labeled.

PROMPT 60 — Strong Lap Filtering Policy (learning vs reporting)
Split lap inclusion into two policies: learning set vs display set. Requirements:

- Learning set excludes: outlaps/inlaps, traffic laps, low distance quality, abnormal lap length, low channel coverage.

- Display set can include more laps but labels them.

- Persist both sets and selection reasons.
  Files: session_trimming.py, pipeline.py stage 7/10, data_quality.py.
  Tests:

- Session with traffic laps: learning excludes them; display still shows with “traffic suspected”.
  Acceptance:

- Theoretical and envelopes are trained only on “clean” laps.

PROMPT 61 — Clear-Lap / Traffic Detection (automatic)
Implement traffic detection robustly. Requirements:

- Detect micro-lifts on straights via speed variance + unexpected a_long dips; correlate with delta spikes.

- Produce per-lap traffic_score and per-segment traffic flags.

- Exclude traffic segments from envelope learning; label in report.
  Files: deltas.py, derived_channels.py, data_quality.py, pipeline.py stage 10, reporting_html.py.
  Tests:

- Synthetic lap with injected lift on straight → flagged as traffic.
  Acceptance:

- Traffic no longer contaminates theoretical and coaching.

PROMPT 62 — Curvature Quality Correction (spike hardening)
Harden curvature estimation used for corners/theoretical. Requirements:

- Compute curvature primarily from smoothed reference line; validate against per-lap curvature.

- Apply constraints: max |dκ/ds|, median filter, outlier removal, fill gaps.

- Persist curvature_quality metrics and gate mu(s) estimation when curvature unreliable.
  Files: reference_line.py, corner_detection.py, theoretical_engine.py, data_quality.py.
  Tests:

- Noisy XY creates curvature spikes; corrected curvature becomes smooth and corner count stable.
  Acceptance:

- Corner detection and mu(s) are stable across laps.

PROMPT 63 — Corner Type Classification (hairpin/chicane/kink/etc.)
Add corner taxonomy to drive better coaching and constraints. Requirements:

- Classify corners from curvature profile + length + entry speed:
  - hairpin, medium, fast, chicane, kink, compound member, setup corner candidate.

- Use type to tune event detectors (thresholds) and coaching templates.
  Files: corner_detection.py, corner_analysis.py, reporting_html.py.

Tests:

- Synthetic curvature shapes map to correct types.
  Acceptance:

- Corner coaching uses correct assumptions per type.

PROMPT 64 — Advice Stability Filter (anti-flip-flop coaching)
Ensure coaching recommendations do not oscillate between laps due to noise. Requirements:

- Aggregate evidence across laps: median event positions, robust deltas.

- Only recommend changes that are stable (low std-dev) and above minimum time threshold.

- Persist stability_score and suppress unstable advice.
  Files: corner_analysis.py coaching generator, pipeline.py, reporting_html.py.
  Tests:

- High-noise session → advice suppressed or softened; stable session → advice appears.
  Acceptance:

- Users don't see contradictory advice session-to-session.

PROMPT 65 — Coaching Confidence Gating (hard thresholds)
Enforce that coaching outputs appear only when confidence is sufficient. Requirements:

- Define per-feature confidence thresholds (events, MRP, brake release, coupling).

- If below threshold: mark unavailable with reason_code, do not output "actionable" text.

- Ensure UI/report shows "why unavailable".
  Files: corner_analysis.py, data_quality.py, reporting_html.py, web/routes.
  Tests:

- Missing yaw_rate → no MRP advice; shown as unavailable.
  Acceptance:

- No low-confidence "fake precision" advice.

PROMPT 66 — Model Versioning + Provenance (debuggable history)
Add robust provenance fields to every persisted analysis. Requirements:

- Store: model_version, solver_version, params_hash, training_data_hash, method_used, generated_at.

- Include these in theoretical_data, coaching_v2, and practice plan outputs.
  Files: pipeline.py, web/models.py, DB migrations (if needed), web/routes.
  Tests:

- Snapshot verifies version fields exist and change only when expected.
  Acceptance:

- Any output is reproducible and traceable.

PROMPT 67 — Telemetry Replay Synchronization (one distance index)
Make replay views consistent across map/speed/corners/events/theoretical. Requirements:

- Standardize a “master distance grid” per lap; everything references indices into it.

- Ensure /corner-replay returns:

  - arrays aligned on same s-grid

  - event markers as indices + s values

  - theoretical ghost aligned to same s

- Remove ad-hoc bisect/index matching; replace with shared helper.
  Files: web/routes/data_v2.py, pipeline.py corner replay assembly, resampling.py helpers, reporting_html.py if needed.
  Tests:

- Unit test verifying identical array lengths and consistent event index mapping.
  Acceptance:

- Hover/crosshair sync is exact; no drift.

— KILLER 10 (requested) —
(Some overlap with the 20 above; these are “product impact first” versions.)

PROMPT 68 — (KILLER) Distance Preflight + Gating (product-grade)
Same as Prompt 01 but add:

- User-facing messages and UI badges (“distance repaired”, “advanced analysis disabled”).

- Telemetry debug section in report: lap length histogram + distance quality summary.
  Files: reporting_html.py, web/routes, pipeline summary.

PROMPT 69 — (KILLER) Single Optimal Engine Rollout Plan (migration-safe)
Implement with migration constraints:

- Keep legacy outputs but add new “optimal_v2” object.

- Provide a feature flag to switch consumers gradually.

- Add backward-compatible DB write/read.
  Acceptance:

- No endpoint breaks; new engine can be enabled per session/user.

PROMPT 70 — (KILLER) Confidence System UX (make trust visible)
Implement Prompt 03 plus:

- Report “Confidence panel” summarizing top 3 blockers and how to fix (e.g., “add IMU”, “do 6 clear laps”).

- API includes `recommended_next_action`.
  Acceptance:

- Users understand why something is missing and what to do.

PROMPT 71 — (KILLER) Traffic Exclusion from Learning (hard rule)
Implement Prompt 14 plus:

- Hard rule: traffic_score above threshold excludes lap from any learning (mu(s), envelopes, calibration, clustering).

- Still show it in replay with a badge.
  Acceptance:

- Theoretical never gets slower due to traffic laps.

PROMPT 72 — (KILLER) Condition Clustering Default (stop blending fiction)
Implement a clustering layer before theoretical generation. Requirements:

- Rule-based clustering first (grip proxy, lap time bands, traffic_score, session phase).
- Output cluster stats + selected cluster id + purity score.
- Theoretical uses selected cluster only; fallback if too few laps.
  Files: pipeline.py stage 10, theoretical_engine.py, data_quality.py, web/models.py, reporting_html.py.
  Tests:
- Synthetic two-regime session: clusters separated and theoretical matches the fast-regime.
  Acceptance:
- Envelopes no longer blend incompatible laps.

PROMPT 73 — (KILLER) Braking + Throttle Event Engine (first-class)
Implement a deterministic event extractor focused on BP/EOB/ThrottleOn/TrackOut:

- Change-point detection with hysteresis and minimum distance duration.
- Output stability metrics (std-dev across laps).
- Feed these events into deltas attribution + coaching targets.
  Files: derived_channels.py, corner_analysis.py, pipeline.py stage 10, web/routes, reporting_html.py.
  Tests:
- Synthetic signals produce correct event positions and stability.
  Acceptance:
- Event-based coaching matches real driver levers.

PROMPT 74 — (KILLER) Feasibility Enforcement on All Targets
Implement a validation layer that checks any target event plan against solver constraints.
Requirements:

- Prevent impossible combinations (e.g., throttle-on before required speed/rotation feasibility).

- If violated: downgrade confidence, adjust targets conservatively, and add warning.
  Files: corner_analysis.py (target generator), solver module, pipeline.py stage 10, reporting_html.py.
  Tests:

- Construct contradictory targets; validator corrects or suppresses with reason.
  Acceptance:

- Targets are always physically plausible.

PROMPT 75 — (KILLER) Limiting Factor Map in UI + Coaching Rationale
Implement :

- Per-corner “why label”: entry-priority vs exit-priority based on limiting factor and following straight length.

- Coaching text must reference limiting factor (e.g., “traction-limited on exit; prioritize rotation before throttle”).
  Files: lap_simulation.py, corner_analysis.py, reporting_html.py.

PROMPT 76 — (KILLER) Model Provenance Contract Everywhere
Implement Prompt 19 plus:

- Add `analysis_run_id` and `input_fingerprint` (hash of normalized input channels).

- Add endpoint to fetch provenance for debugging.
  Files: web/models.py, web/routes.

PROMPT 77 — (KILLER) Deterministic Output Snapshot Suite
Implement:

- Golden snapshot tests for:
  - /speed-trace payload
  - /corners payload (including events)
  - theoretical_data payload
- CI gate: any change requires intentional snapshot update.
  Files: tests/, web/routes tests, pipeline integration tests.

Acceptance:

- You can refactor safely without accidental behavior drift.

.

prompt1 + prompt2 + prompt3 V

prompt4 + prompt10 + prompt11 V

prompt6 V

prompt8 V

prompt7 V

prompt9 V

prompt13 + prompt14 V

prompt15 V

prompt16 V

prompt17 V

prompt18 V

prompt19 + prompt20 + prompt21 V

prompt22 V

prompt23 + prompt24 + prompt25 V

prompt30 + prompt31 V

prompt27 + prompt28 + prompt29 V

prompt32 V

prompt33 V

prompt5 V

prompt51 + prompt66 + prompt76 V

prompt48 + prompt68 + prompt62 V

prompt52 + prompt54 V

prompt61 + prompt71 + prompt60 V

prompt42 + prompt72 + prompt50 + prompt70 V

prompt43 + prompt53 V

prompt49 + prompt69 V

prompt55 + prompt75 + prompt56 V

prompt57 V

prompt73 + prompt34 + prompt35 V

prompt36 + prompt37 V

prompt63 + prompt38 + prompt39 V

prompt46 + prompt74 V

prompt59 + prompt58 V

prompt40 + prompt64 + prompt65 V

prompt41 + prompt47 V

prompt45 + prompt44

prompt67

prompt77